Министерство образования
и науки Российской Федерации



УРАЛЬСКИЙ
ФЕДЕРАЛЬНЫЙ
УНИВЕРСИТЕТ
имени первого Президента
России Б. Н. Ельцина

В.Ю. Кара-Ушанов

МОДЕЛЬ «СУЩНОСТЬ – СВЯЗЬ»

Электронное текстовое издание

Учебное пособие предназначено для студентов любых направлений подготовки, изучающих в дисциплинах «Базы данных», «Управление данными» методологию проектирования информационных систем.

Подготовлено кафедрой технической физики.

В учебном пособии обсуждаются основные концепции моделирования данных, основанного на методе «сущность – связь». Демонстрируется технология применения методов разработки информационной модели в инструментальной среде популярной CASE-системы ERwin. На конкретном содержательном примере закрепляются навыки практической работы по построению информационной модели.

**Екатеринбург
2017**

ОГЛАВЛЕНИЕ

ВВЕДЕНИЕ 3
1. ОСНОВНЫЕ КОНЦЕПЦИИ МОДЕЛИ «СУЩНОСТЬ – СВЯЗЬ» 4
2. РОЛЬ АБСТРАКЦИИ В СИСТЕМАТИЗАЦИИ ДАННЫХ 7
2.1. Абстракция обобщения 7
2.2. Абстракция агрегации 8
2.3. Иерархия обобщения и агрегации 10
3. ЭЛЕМЕНТЫ ТЕОРИИ ДАННЫХ 12
3.1. Домены и атрибуты 12
3.2. Атрибуты и сущности 14
3.3. Сущности и связи 16
3.4. Ключи 16
4. СИНТАКСИС ER-МОДЕЛИ В НОТАЦИИ IDEF1X 20
4.1. Синтаксис сущностей в IDEF1X 20
4.2. Синтаксис атрибутов в IDEF1X 21
4.3. Синтаксис связей в IDEF1X 21
4.4. Ограничения ссылочной целостности 31
5. ПОСТРОЕНИЕ ИНФОРМАЦИОННОЙ МОДЕЛИ 36
5.1. Создание сущностей 38
5.2. Создание связей 41
5.3. Создание атрибутов 48
5.4. Определение первичных и альтернативных ключей 51
5.5. Детализация ER-модели 55
ЗАКЛЮЧЕНИЕ 61
БИБЛИОГРАФИЧЕСКИЙ СПИСОК 62

ВВЕДЕНИЕ

Когда специалист приступает к разработке информационной системы, на этапе предпроектного исследования и анализа предметной области он осуществляет сбор, накопление и систематизацию актуальной информации. Эта информация представлена множеством информационных элементов. Проблема состоит в том, чтобы упорядочить эту информацию, классифицировать и структурировать информационные элементы. Естественным средством решения этой проблемы является анализ семантики данных предметной области и систематизация представлений аналитика о структуре данных в виде модели данных. Поэтому на ранних этапах жизненного цикла информационной системы большое значение имеет моделирование данных [1–4].

Модель данных является визуальным представлением структуры данных и бизнес-правил с точки зрения человека или компьютера (СУБД). Информационная модель зависит от уровня абстракции представления структуры данных конкретной предметной области. В связи с этим принято различать **логическую** и **физическую** модели. В отечественной литературе традиционно используются, соответственно, понятия «инфологическая» и «даталогическая» модели данных. Инфологическая модель представляет собой описание предметной области, основанное на анализе семантики объектов и явлений, выполненное без ориентации на использование в дальнейшем программных или технических компьютерных средств. Термин «инфологическая» в данном случае означает модель, ориентированную на человека; речь идет о средстве для выражения и передачи понимания того, что собой представляет предметная область базы данных. Даталогическая модель представляет собой описание моделируемых данных с учетом тех структурных и процедурных возможностей, которые предоставляет компьютерная система. Термин «даталогическая» означает СУБД-зависимую модель, ориентированную на человека. В данном учебном пособии мы ограничимся только освоением методологии инфологического моделирования.

1. ОСНОВНЫЕ КОНЦЕПЦИИ МОДЕЛИ «СУЩНОСТЬ – СВЯЗЬ»

Модельные представления, основанные на анализе семантики данных, иногда называют семантическими моделями. Одним из распространенных средств спецификации модельных представлений этого типа является т.н. «**модель сущность – связь**» (Entity – Relationship Model) [2–5].

Модель «сущность – связь» (ER-модель) была предложена П. Ченом в 1976 году [5] как средство «ручного» проектирования баз данных. Модель «сущность – связь» представляет собой набор концепций, используемых для описания логической структуры базы данных.

Базис понятий семантического моделирования в общем случае включает:

- определение объекта-экземпляра;
- определение объекта-типа;
- определение связи между объектами;
- определение свойства объекта;
- определение идентифицирующего свойства объекта.

Для модели «сущность – связь» базовыми являются понятия:

- сущность;
- связь;
- атрибут.

Модель «сущность – связь» основана на диаграммной технике. Для представления различных аспектов структуры данных (объектов, свойств объектов, связей между объектами, свойств связей и других) используются графические средства.

За многолетнюю историю использования модель «сущность – связь» претерпела различные модификации, известны несколько ее нотаций. Наиболее популярной в настоящее время является нотация, в большей степени приближенная к реальному процессу проектирования баз данных, т. н. **методология информационного моделирования IDEF1X** [7–10].

В ER-модели абстрактным объектам реальной действительности соответствует понятие «сущность».

Сущность – это абстрактный объект, который в конкретном контексте имеет независимое существование.

Различают понятия «тип сущности» и «экземпляр сущности».

Тип сущности (в дальнейшем просто сущность) представляет собой объект-тип, результат абстракции обобщения множества однородных объектов-экземпляров реальной действительности с одинаковыми свойствами.

Сущность имеет семантически значимое имя, как правило, имя существительное, например: «Студент», «Преподаватель», «Сотрудник», «Отдел», «Компьютер, «Книга» и т. д.

Экземпляр сущности соответствует объектам реальной действительности. Это конкретные персоны студентов, преподавателей, сотрудников, конкретные отделы на предприятии, конкретные компьютерные устройства, переплеты книг и т. д. Экземпляры сущностей уникальны, в природе не бывает двух одинаковых объектов. Следовательно, экземпляр сущности может быть идентифицирован уникальным образом.

Связь – это ассоциация сущностей или отношение между сущностями.

Различают понятия «тип связи» и «экземпляр связи».

Тип связи (в дальнейшем просто связь) представляет собой отношение между типами сущностей.

Связь имеет семантически значимое имя, как правило, в форме глагола, например: «Преподаватель» «Ведет» «Дисциплину», «Отдел» «Включает» «Сотрудника» и т. д.

Экземпляр связи представляет собой отношение между экземплярами сущностей, например: Иванов «Ведет» Математику, Отдел 101 «Включает» Петрова и т. д.

Связи обладают свойствами:

- вид (категориальная, идентифицирующая, неидентифицирующая);
- степень (унарная, бинарная, тернарная, N-арная);
- кардинальность;
- внешний ключ сущности.

Атрибут – это свойство сущности или связи.

Атрибутам как спецификаторам свойств сущностей или связей присваиваются семантически значимые имена, как правило, в форме существительного.

Атрибутами сущности «Сотрудник» являются: *табельный номер, фамилия, имя, отчество, дата рождения* и другие персональные характеристики.

Атрибутами тернарной связи «Экзамен» между сущностями «Студент», «Преподаватель», «Дисциплина» являются *идентификационные номера студента, преподавателя и дисциплины*, а так же *время и место проведения экзамена*.

С понятием атрибута связаны понятия:

- домен атрибута;
- зависимости между атрибутами;
- потенциальный ключ сущности;
- детерминант и другие.

2. РОЛЬ АБСТРАКЦИИ В СИСТЕМАТИЗАЦИИ ДАННЫХ

При моделировании статических свойств предметной области, представленных главным образом абстракциями состояния или абстрактными объектами, существенное значение имеет интеграция понятий, представляющих объекты. Одна из главных целей моделирования, основанного на анализе семантики данных, заключается в том, чтобы поддерживать такую гибкость интерпретации, которая позволяет рассматривать любой объект (сущность), свойство объекта (атрибут) и отношение (связь) между объектами как разные роли, которые может играть одни и тот же объект в зависимости от контекста его рассмотрения.

База данных, как правило, состоит не из независимых объектов: обычно некоторые объекты могут быть образованы из других объектов. В результате между объектами устанавливаются отношения, которые должны быть систематизированы. В процессе построения модели, основанной на анализе семантики, систематизация данных конкретной предметной области осуществляется на основе методов абстракции – методов **обобщения** и **агрегации** [6], которыми люди интуитивно пользуются в обыденной жизни.

2.1. Абстракция обобщения

> **Обобщением** называется абстракция, которая позволяет трактовать класс различных подобных объектов (сущностей) как один поименованный обобщенный тип объекта.

Обобщение (или генерализация) порождает новый объект путем сведения к минимуму различия между сущностями и выделения их общих характеристик. Например, в результате обобщения объектов «Преподаватель», «Научный сотрудник», «Инженер» может быть создан объект «Сотрудник». В общем случае, если класс однородных объектов $\{O_1,O_2,O_3,\ldots,O_n\}$ можно обобщить до объекта-типа O, то O_i называются **категориями**, а O – **обобщением**. Установить, имеется ли между объектами отношение обобщения

можно только на основе анализа семантики данных. Именно семантически значимые имена объектов несут на себе смысловое значение обобщения.

В результате обобщения между объектами устанавливается отношение (связь) с семантикой «есть» (is a): «Преподаватель» есть «Сотрудник», «Инженер» есть «Сотрудник» и т. д.

Обобщение позволяет соотнести множество знаков (экземпляров) или множество типов с одним обобщенным типом объекта. В связи с этим различают обобщения:

- знак – тип;
- тип – тип.

Обобщение экземпляров сущности (знаков) до типа сущности называется **классификацией**. Например, множество экземпляров конкретных персон сотрудников {Сотрудник 1001, Сотрудник 1002, Сотрудник 1003, …} может быть обобщено до типа сущности «Сотрудник». Процесс, обратный классификации, называется **экземпляризацией** (порождением реализаций).

Установление отношений между типами сущностей (обобщение «тип-тип») называется **обобщением** или **генерализацией**. Процесс, обратный обобщению, называется **специализацией**. Специализация основана на увеличении различий между категориями за счет выделения их отличительных характеристик.

Обобщенный тип обладает всеми свойствами, общими для базовых знаков или типов. Или, иначе, – все свойства обобщенного типа могут быть унаследованы базовыми типами (категориями).

2.2. Абстракция агрегации

> **Агрегацией** называется абстракция, которая позволяет рассматривать связь (отношение) между объектами как новый объект.

Например, объекты «Табельный номер сотрудника», «Фамилия», «Дата рождения», «Адрес»… можно агрегировать в отношение с семантически значимым именем «Сотрудник». В общем случае, если отношение объектов

(O_1, O_2, O_3, ..., O_n) агрегирует объекты O_i в O, то O_i называются **компонентами (атрибутами)**, а O – **отношением (сущностью)**. Агрегация объектов-компонентов O_i в отношение O возможна на основе анализа семантики данных. Именно семантически значимые имена объектов несут на себе смысловое значение агрегации.

В результате агрегации между объектами устанавливается отношение (связь) с семантикой «есть часть» (is part of): «Фамилия» есть часть «Сотрудник», «Адрес» есть часть «Сотрудник» и т. д.

Агрегация может применяться:

– на уровне типов;

– на уровне знаков.

В ER-модели тип сущности (сущность) представляет собой агрегат (отношение) атрибутов (компонентов) – спецификаторов свойств сущности. Например, сущность «Сотрудник» является отношением других объектов: «табельный номер сотрудника», «фамилия», «дата рождения», «адрес»,…, которые в данном контексте играют роль типов свойств – атрибутов. Такие свойства, присущие всему классу подобных объектов, называются **интенсиональными** свойствами.

Связь (тип связи) между сущностями также представляет собой отношение (агрегат) типов сущностей: «Проектирование» («Проект», «Сотрудник»).

Экземпляр сущности «Сотрудник» так же является агрегатом, агрегатом свойств-знаков, например (1001, Иванов, 25.04.1980, Мира 3-45). Каждый экземпляр связи «Проектирование» как агрегат дает представление о том, какие сотрудники участвуют в разработке тех или иных проектов, например, (Проект1, 1001). Подобные свойства-знаки называются **экстенсиональными** свойствами, они присущи экземпляру сущности или связи.

Методы абстракции обобщения и агрегации могут быть полезными для систематизации данных на этапе предпроектного анализа предметной области.

Обобщение и агрегация представляют разные контексты анализа одних и тех же информационных элементов, взаимно дополняют друг друга, обогащая семантическое содержание данных. Любой объект, например «Сотрудник» (рис. 1), может одновременно рассматриваться в отношении обобщения с одними объектами и в отношении агрегации – с другими.

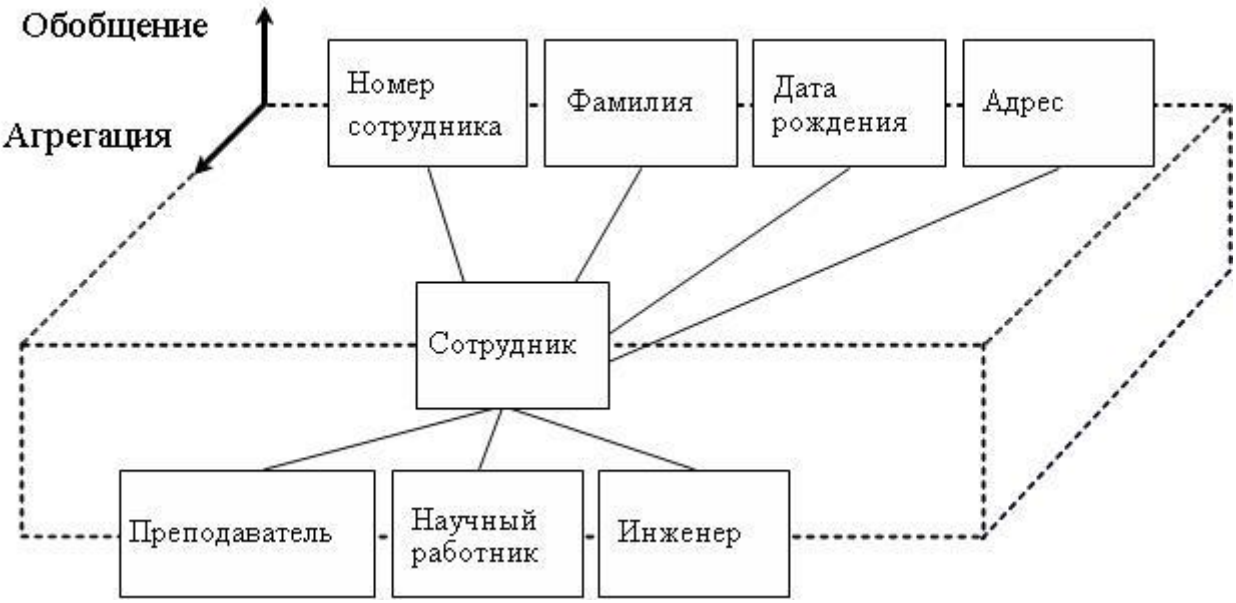


Рис. 1. Абстракции обобщения и агрегации

Обобщение и агрегация представляют разные аспекты систематизации данных. Если обобщение позволяет **классифицировать** объекты, то агрегация раскрывает **структуру** объектов.

И обобщение, и агрегация могут быть применены иерархическим образом. При этом классификация объектов-типов (сущностей) может быть выражена **иерархией обобщения**, а структура данных предметной области – **иерархией агрегации**.

2.3. Иерархия обобщения и агрегации

Классификация типов сущностей может быть выражена иерархией обобщения (рис. 2). Сущности в иерархии обобщения связаны родовидовыми отношениями («тип – подтип») со степенью один к одному. Любой экземпляр категориальной сущности является экземпляром обобщающей сущности и в то же время имеет отличительные свойства-признаки.

Иерархия обобщения допускает совместное использование одной категориальной сущности несколькими обобщающими сущностями, например «Студент». При этом категориальная сущность наследует свойства всех своих родителей.

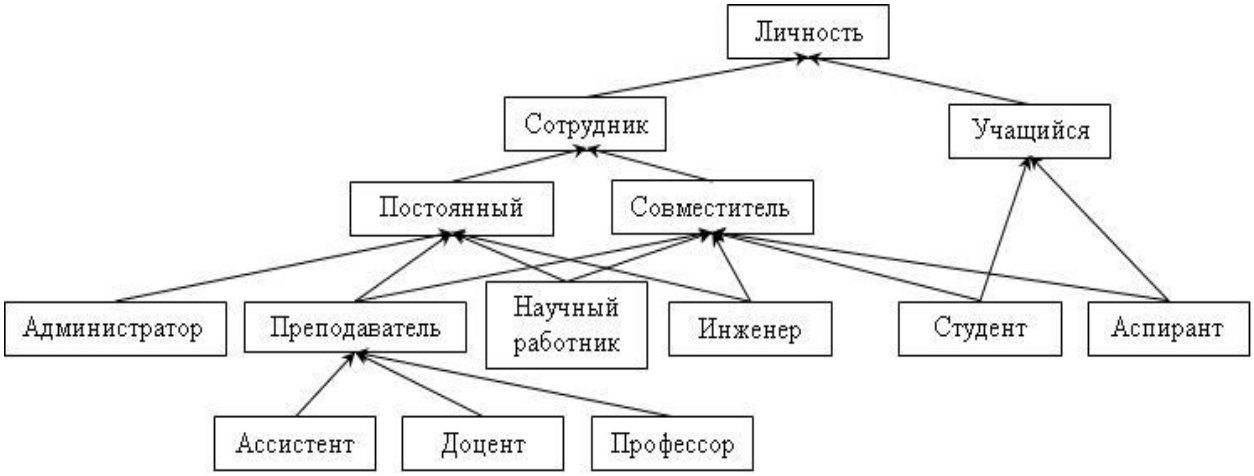


Рис. 2. Иерархия обобщения

Структура типов объектов может быть выражена иерархией агрегации (рис. 3). Роль, которую играет объект, зависит от контекста.

Если интерес представляет «Сотрудник», тогда он будет рассматриваться как сущность, «Образование» как отношение (связь с другими сущностями), а «Комната» как атрибут сущности

Если интерес как объект представляет «Образование», тогда «Сотрудник» будет рассматриваться как атрибут этого объекта.

Если интерес как объект представляет «Комната», тогда «Сотрудник» будет рассматриваться как отношение.

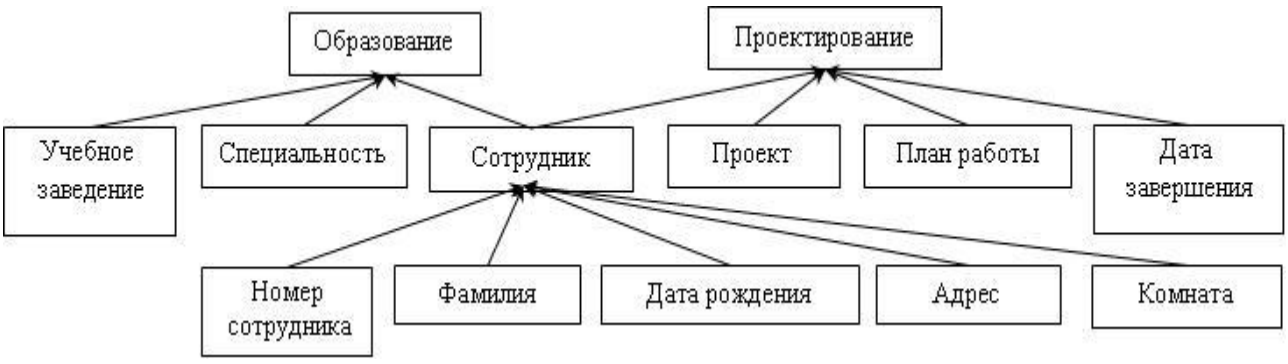


Рис. 3. Иерархия агрегации

3. ЭЛЕМЕНТЫ ТЕОРИИ ДАННЫХ

Теоретической основой моделирования данных является теория множеств, в терминах которой можно описать любую модель и ее конструктивные элементы. В математике множество – это абстрактное понятие, но в теории данных понятие «множество» наполняется конкретным семантическим содержанием.

Базовые понятия модели данных могут быть определены как множества. Например, базовые понятия модели «сущность – связь» атрибут, сущность, связь могут рассматриваться как множества, соответственно, множество элементов данных скалярного типа, множество экземпляров сущности и множество экземпляров связи.

Зависимости между компонентами структуры данных реальной действительности в теории множеств принимают вид соответствий и отображений множеств. Наиболее распространенным частным случаем соответствия является функциональное соответствие или отображение $f:X \rightarrow Y$, которое каждому элементу x множества-прообраза X ставит в соответствие **один** элемент y множества-образа Y.

3.1. Домены и атрибуты

Каждому атрибуту как спецификатору свойств сущности и связи можно поставить в соответствие множество однородных элементов данных (знаков), например, множество целых чисел от 10 до 20, строки символов длиной до 20 и т. д. Подобные однородные множества в моделировании данных носят название доменов.

> **Домен** – это семантическое понятие. Домен можно рассматривать как подмножество значений некоторого типа данных имеющих определенный смысл.

Домен характеризуется следующими свойствами:

– имеет уникальное имя;

– определен на некотором простом (скалярном) типе данных или на другом домене;

– может иметь некоторое логическое условие, позволяющее описать подмножество данных, допустимых для данного домена;

– может быть задан перечислением множества допустимых элементов данных;

– несет определенную смысловую нагрузку.

Примеры доменов:

– домен «зарплата» – множество вещественных чисел, превышающих МРОТ (минимальный размер оплаты труда);

– домен «образование» – множество строк символов, заданное перечислением {начальное, среднее общее, среднее специальное, высшее};

– домен «фамилия» – множество строк символов не длиннее 20;

– домен «оценка» – множество целых чисел в интервале от 2 до 5;

– домен «пол» – множество данных логического типа {мужской, женский};

– домен «номер» – множество положительных целых чисел.

По отношению к атрибутам домены играют роль области определения. Соответствие между доменами и атрибутами примерно такое же, как между типами данных и переменными в языках программирования. Атрибут в такой интерпретации – это предметная переменная, определенная на домене.

Однако, домен и тип данных это не тождественные понятия. Домен представляет собой подмножество допустимых элементов данных некоторого типа, имеющее семантически значимое имя. Отличие домена от подмножества данных некоторого типа состоит именно в том, что **домен отражает семантику**, определенную предметной областью. Может быть несколько доменов, совпадающих как подмножества, но несущие различный смысл. Например, на множестве неотрицательных целых чисел могут быть определены

домены «табельный номер», «стаж», «отпуск», «возраст», но смысл этих доменов будет различным, и это будут различные домены.

Понятие домена помогает **правильно моделировать** предметную область, **исключая некорректные сравнения** семантически разнородных данных. Некорректно, с логической точки зрения, сравнивать значения из различных доменов, даже если они имеют одинаковый тип. В этом проявляется смысловое ограничение доменов.

Домены могут представлять собой комбинации других доменов, образованные по принципу агрегации. Например, домен «дата» может состоять из подчиненных доменов «день», «месяц», «год», а домен «адрес» может состоять из доменов «город», «улица», «номер дома».

На одном и том же домене могут быть определены несколько атрибутов. Например, на домене «выплаты» могут быть определены атрибуты «зарплата», «премия», «пособие», «ссуда» и т. д. Однако, любой атрибут может быть определен только на одном домене.

> Итак, областью определения атрибутов являются специфические множества элементов данных или домены.

3.2. Атрибуты и сущности

Атрибуты по отношению к сущности играют роль спецификаторов ее свойств

Тип сущности E является обобщением экземпляров сущности e_i, которые так же образуют множество:

$$E = \{e_1, e_2, e_3, \ldots, e_n\}.$$

Экземпляр сущности представляет конкретный объект реальной действительности. Какую роль играют атрибуты в идентификации экземпляров сущности? Множество экземпляров сущности определяется ассоциативно множествами значений атрибутов (доменами) в результате функционального (однозначного) отображения множеств значений атрибутов (доменов) на множество экземпляров сущности.

Если сущность моделирует агрегированный объект, например, *Сотрудник (табельный номер, фамилия, ...)*, или в общем случае $E(A_1, A_2, \ldots, A_n)$, то областью определения соответствующего отображения f множеств значений атрибутов на множество экземпляров сущности будет, по крайней мере, декартово произведение множеств значений атрибутов A_i:

$$f : A_1 \times A_2 \times \cdots \times A_n \to E .$$

Декартово произведение – это теоретико-множественная операция, которая формирует результат из множеств-операндов, например $A=\{a_1, a_2\}$ и $B=\{b_1, b_2, b_3\}$, по следующему образцу:

$$A \times B = \{ \langle a_i, b_j \rangle \mid a_i \in A \wedge b_j \in B \} =$$

$$= \{ \langle a_1, b_1 \rangle, \langle a_1, b_2 \rangle, \langle a_1, b_3 \rangle, \langle a_2, b_1 \rangle, \langle a_2, b_2 \rangle, \langle a_2, b_3 \rangle \}$$

Итак, в терминах теории множеств тип сущности определен на декартовом произведении множеств значений атрибутов:

$$A_1 \times A_2 \times \cdots \times A_n = \{ \langle a_1, a_2, \cdots, a_n \rangle \mid a_1 \in D_1 \wedge a_2 \in D_2 \wedge \cdots \wedge a_n \in D_n \}$$

А экземпляр сущности e_i представляет собой кортеж декартового произведения вида $\langle a_1, a_2, \cdots, a_n \rangle$. Элементами кортежа являются значения атрибутов сущности a_j, принадлежащие доменам D_j.

Декартово произведение определяет множество **возможных** комбинаций значений атрибутов в кортеже, как например, в нашем случае:

$$\{\langle 1001, Иванов, ... \rangle \langle 1001, Петров, ... \rangle ... \langle 1002, Иванов, ... \rangle \langle 1002, Петров, ... \rangle ...\} .$$

Однако не все такие комбинации являются допустимыми вследствие **ограничений целостности объекта** (сущности), обусловленных семантикой связей между атрибутами. Например, семантика понятия «табельный номер», как уникального идентификатора персоны сотрудника, дает основания для исключения двух из приведенных кортежей. Поэтому областью определения множества экземпляров сущности является подмножество декартового произведения – **отношение**.

Итак, областью определения сущности является отношение, определенное на доменах, ассоциированных с ее атрибутами. Каждый кортеж такого отношения представляет экземпляр сущности.

3.3. Сущности и связи

Тип связи *R* является обобщением экземпляров связей r_i, которые так же образуют множество:

$$R = \{r_1, r_2, r_3, \ldots, r_k\}.$$

Тип связи определен на декартовом произведении множеств экземпляров сущностей:

$$E_1 \times E_2 \times \cdots \times E_k = \{ \langle e_1, e_2, \cdots, e_k \rangle \mid e_1 \in E_1 \wedge e_2 \in E_2 \wedge \cdots \wedge e_k \in E_k \}$$

на результате сцепления (конкатенации) кортежей, представляющих экземпляры сущностей.

Экземпляр связи r_i представляет собой кортеж декартового произведения вида $\langle e_1, e_2, \cdots, e_k \rangle$.

В терминах теории множеств областью определения типа связи является отношение – подмножество декартового произведения множеств экземпляров связываемых сущностей.

Множество допустимых кортежей определяется действующими в предметной области семантическими условиями или бизнес-правилами, которые играют роль **ограничений целостности связей (ссылочной целостности)**.

Наиболее распространенным типом отношения (связи) сущностей является бинарное отношение, образованное бинарными кортежами. Бинарные отношения бывают следующих типов: «один к одному», «один ко многим» и «многие ко многим».

3.4. Ключи

Атрибуты используются не только для описания свойств сущностей и связей, но и для идентификации их экземпляров. Экземпляр сущности (или связи) однозначно идентифицируется набором **всех** ее атрибутов, образующих

кортежи. Однако в большинстве случаев полный набор атрибутов является избыточным для целей идентификации. Это так потому, что между атрибутами имеются зависимости, обусловленные семантикой данных предметной области, следовательно, экземпляр сущности (или связи) может быть идентифицирован подмножеством ее атрибутов. В связи с этим в теории данных вводится понятие потенциального ключа сущности (или связи).

Потенциальным ключом называется подмножество атрибутов (типа сущности или связи), которое **функционально полно** определяет значение любого атрибута.

Термин «функциональная зависимость» означает однозначную зависимость от ключа.

Термин «функционально полная зависимость» означает, что никакое подмножество ключа не может выступать в роли ключа.

Например, между атрибутами сущности «Сотрудник» возможны связи, представленные на рис. 4. На роль потенциального ключа претендуют «табельный номер» и комбинация «серия паспорта» и «номер паспорта».

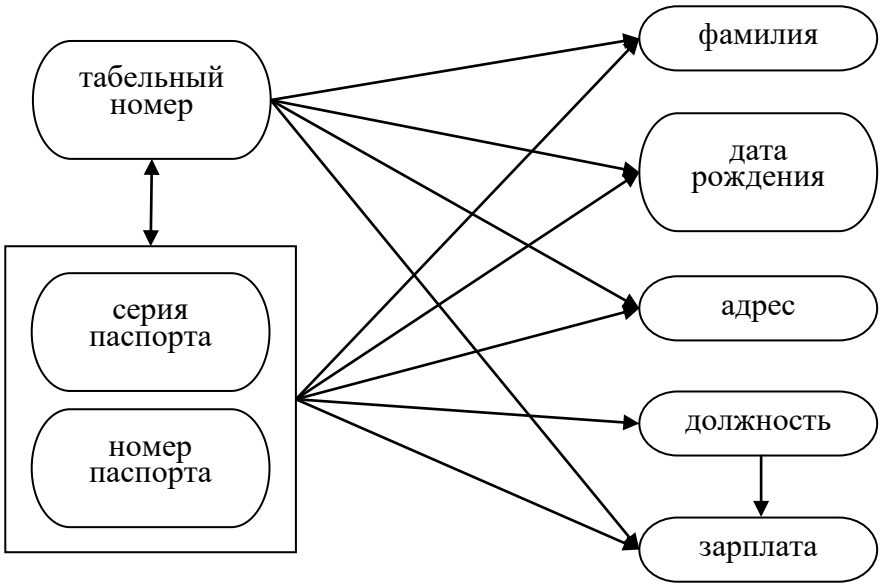


Рис. 4. Схема функциональных зависимостей

Потенциальный ключ может быть:

– **простой** – состоит из одного атрибута, например, «табельный номер»;

– **составной** – состоит из нескольких атрибутов, например, («серия паспорта», «номер паспорта»).

Среди потенциальных ключей выбирают по соображениям простоты манипулирования один, **первичный ключ**, остальные ключи называются **альтернативными**. В иллюстративном примере преимущество следует отдать «табельному номеру» в качестве первичного ключа. Тогда составной ключ («серия паспорта», «номер паспорта») становится альтернативным ключом.

Основными свойствами потенциального ключа являются:

– **уникальность** – не может быть двух и более экземпляров сущности с одинаковыми значениями ключа;

– **неизбыточность** – никакое подмножество ключа не может выступать в роли ключа;

– **обязательность** (определенность) – ни при каких условиях атрибуты ключа не могут принимать неопределенные (Null) значения.

Атрибут «должность», не будучи ключом, тем не менее, однозначно определяет другой атрибут – «зарплата». Такие атрибуты называются **детерминантами**. Ключ является частным случаем детерминанта.

Потенциальный ключ, чья идентифицирующая роль обусловлена семантикой данных, иногда называют **естественным ключом**.

В качестве альтернативы часто используется т. н. **искусственный (суррогатный) ключ**. Искусственный ключ – это дополнительный атрибут, никак не связанный с содержанием данных. Его значениями являются условные порядковые номера экземпляров сущности. Искусственный ключ является элементом управления реализацией семантической модели в виде базы данных. Его преимущества перед естественным ключом в этом качестве проявляется в случае реструктуризации базы данных.

Другим элементом управления реализацией модели является т. н. **неуникальный индекс (инверсионный вход)**. Статус индекса присваивается

атрибуту или группе атрибутов, которые, не будучи ключом, позволяют получить доступ к экземплярам сущности в упорядоченном по индексу виде.

Важной характеристикой связи между старшей сущностью (родительской или обобщением) и младшей (потомком или категорией) является внешний ключ.

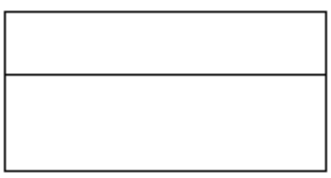
> **Внешний ключ** младшей сущности – это первичный ключ старшей сущности, мигрировавший в младшую для моделирования связи.

4. СИНТАКСИС ER-МОДЕЛИ В НОТАЦИИ IDEF1X

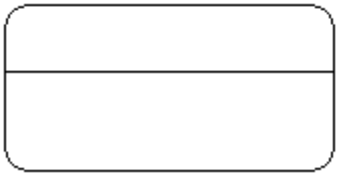
Наиболее распространенной нотацией моделирования сущностей и связей является нотация IDEF1X, которая подробно описана в литературе [7–10]. Знание синтаксических правил модельных представлений является обязательным условием грамотного и полноценного моделирования структуры данных. Условности отображения конструктивных элементов модели призваны отобразить на диаграмме как можно больше смысла и значения (семантики) данных моделируемой предметной области, чтобы модель по возможности максимально полно отображала реальное состояние моделируемого объекта.

4.1. Синтаксис сущностей в IDEF1X

1. Сущность на ER-диаграммах изображается блоками:

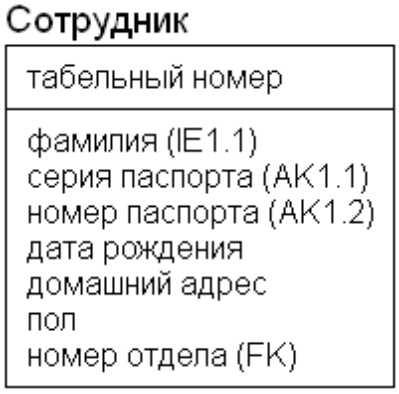


независимая



зависимая

2. Сущность имеет уникальное имя. Именем сущности является имя существительное. Имя сущности указывается над блоком («Сотрудник»).



3. Сущность обладает одним или несколькими атрибутами, которые однозначно идентифицируют каждый ее экземпляр (первичный ключ «табельный номер», альтернативный ключ «серия паспорта, номер паспорта»).

4. Сущность обладает одним или несколькими атрибутами, которые либо принадлежат ей, либо наследуются через отношение с другими сущностями (внешний ключ «номер отдела»).

4.2. Синтаксис атрибутов в IDEF1X

1. Атрибуты изображаются в виде списка их имен и меток внутри блока сущности:

2. Каждый атрибут имеет уникальное имя. Именем атрибута является имя существительное.

3. Атрибуты первичного ключа («табельный номер») располагаются в начале списка и отделяются от других атрибутов горизонтальной чертой.

4. Атрибуты альтернативного ключа («серия паспорта», «номер паспорта») могут быть помечены меткой $AKn.m$, где n – номер ключа, m – номер атрибута в ключе.

5. Внешний ключ («номер отдела») имеет метку FK и занимает свое место в сущности-потомке после установления связи с родительской сущностью в результате миграции ее первичного ключа.

4.3. Синтаксис связей в IDEF1X

1. Связи бывают двух типов:

1.1. Связи в иерархии агрегации (отношения связи родитель-потомок).

1.2. Связи в иерархии обобщения (родовидовые, категориальные связи).

2. Связи родитель-потомок бывают:

2.1. Унарные (рекурсивные) – связь сущности самой с собой.

2.2. Бинарные – связи между двумя сущностями, в том числе:

2.2.1. Связь «один ко многим»:

2.2.1.1. Идентифицирующие.

2.2.1.2. Неидентифицирующие.

2.2.2. Связь «многие ко многим».

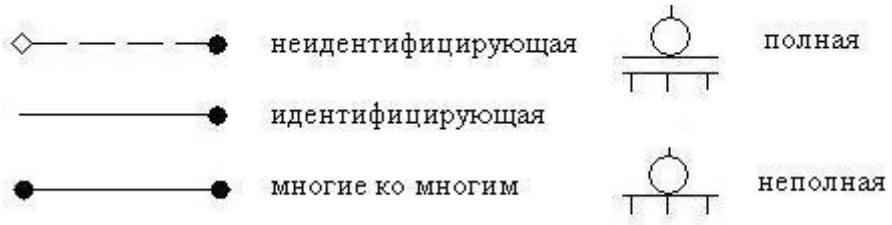
2.3. N-арные связи моделируются при помощи ассоциативных сущностей.

3. Категориальные связи бывают:

3.1. Полные.

3.2. Неполные.

4. Связи изображаются линиями (дугами):

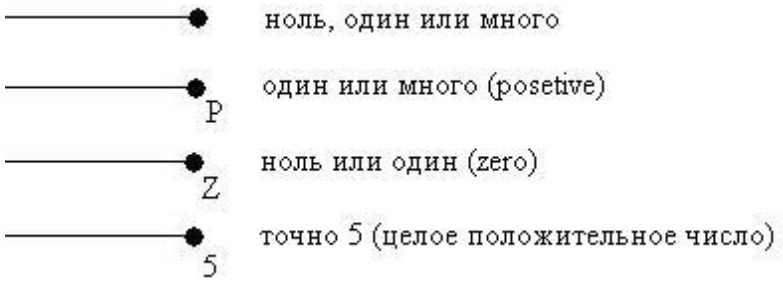


5. В числе атрибутов родовой (обобщающей) сущности должен присутствовать **атрибут-дискриминатор** сущностей-категорий. Его имя («тип») указывается рядом с кружком изображения связи.

6. Связи присваивается семантически значимое уникальное имя. Именем связи является глагольная форма. Связи «один ко многим» именуются со стороны родительской сущности. Связи «многие ко многим» – в обоих направлениях. Категориальные связи не именуются.

7. Связь между сущностями моделируется при помощи внешнего ключа. **Внешний ключ** – это первичный ключ родительской или обобщающей сущности, мигрировавший в сущность-потомок или категориальную сущность, соответственно.

8. Для связи может быть указана **мощность** (**кардинальность**), которая показывает, сколько экземпляров младшей сущности соответствуют одному экземпляру старшей сущности:



9. Для уточнения семантики связи родитель-потомок в некоторых случаях можно, а в некоторых бывает необходимо дополнить базовое имя атрибута внешнего ключа **именем роли (**функциональным именем), которую он играет в сущности-потомке как ключ связи.

10. Изображение связи может сопровождаться индикаторами ограничений ссылочной целостности при выполнении вставки, обновления и удаления экземпляров связанных сущностей.

Рассмотрим примеры связей.

Допустим, объектом моделирования является некая организация, которая состоит из отделов. Причем возможно создание отделов, временно не укомплектованных сотрудниками. Сотрудники, работающие в организации, могут быть временно не приписаны ни к одному отделу. В этом случае между сущностями «Отдел» и «Сотрудник» имеется **неидентифицирующая** связь (рис. 5).



Рис. 5. Неидентифицирующая связь

Связь между родительской сущностью и сущностью-потомком называется **неидентифицирующей**, если экземпляры сущности-потомка идентифицируются ее собственным первичным ключом, а не ключом связи с родительской сущностью (внешним ключом).

Внешний ключ сущности-потомка **не входит в состав ее первичного ключа**, используется только для моделирования связи с родительской сущностью, но не для идентификации экземпляров.

Сущность, которая идентифицируется своими собственными средствами (первичным ключом), а не по связи с родительской сущностью (внешним ключом), называется независимой сущностью. Сущности «Отдел» и «Сотрудник» являются независимыми сущностями.

Направление неидентифицирующей связи устанавливается от родительской сущности к сущности-потомку.

Кардинальность связи «Включает» соответствует типу «ноль, один или много» (на диаграмме не изображается как умолчание).

Другой пример. Допустим, объектом моделирования является разработка проектов в проектной организации. Причем каждый проект имеет уникальный номер и название (тему). Проект состоит из заданий (этапов). Каждое задание имеет уникальный номер только в пределах своего проекта. В этом случае между сущностями «Проект» и «Этап» имеется **идентифицирующая** связь (рис. 6).



Рис. 6. Идентифицирующая связь

> Связь между родительской сущностью и сущностью-потомком называется **идентифицирующей**, если экземпляры сущности-потомка идентифицируются не только ее собственным первичным ключом, но и ключом ее связи с родительской сущностью (внешним ключом).

Внешний ключ сущности-потомка **входит в состав ее первичного ключа**, используется не только для моделирования связи с родительской сущностью, но и для идентификации экземпляров.

Сущность, которая идентифицируется не только своими собственными средствами (первичным ключом), но и по связи с родительской сущностью (внешним ключом), называется **зависимой сущностью**. Сущности «Проект» является независимой, сущность «Этап» является зависимой сущностью.

Направление идентифицирующей связи устанавливается от родительской сущности к сущности-потомку.

Кардинальность связи «Состоит из» соответствует типу «один или много» и обозначается меткой *P*.

Внешний ключ сущности-потомка («номер проекта») представлен своим базовым именем.

Еще один пример. Допустим, сотрудники некоторой организации могут работать в качестве постоянных (штатных) сотрудников, в качестве совместителей или в качестве внешних консультантов. У всех этих категорий сотрудников есть некоторые общие, персональные характеристики, которые могут послужить основанием для их обобщения и создания новой обобщающей сущности. Для моделирования такого рода отношений между сущностями-категориями «Постоянный», «Совместитель», «Консультант» и обобщающей сущностью «Сотрудник» используется **категориальная** связь (рис. 7).

Категориальные связи не именуются.

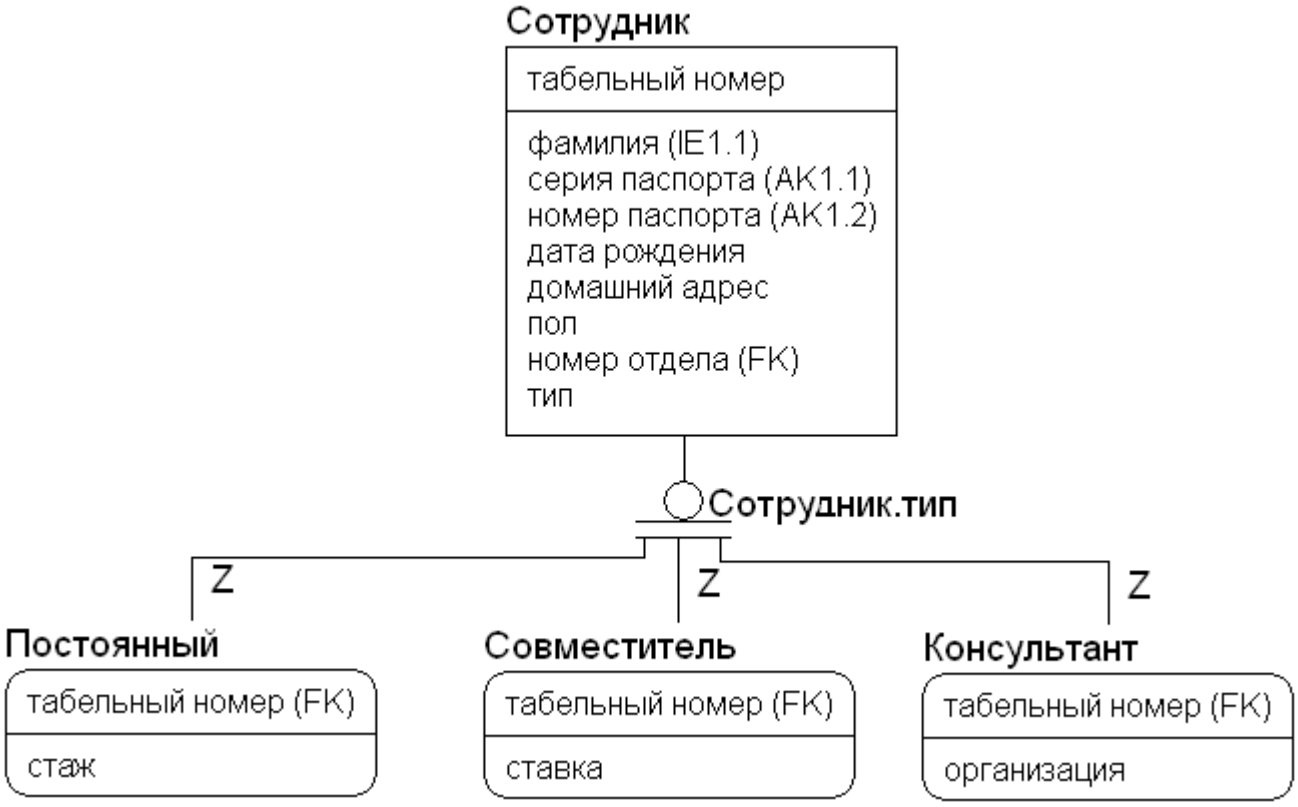


Рис. 7. Категориальная полная связь

Между обобщающей сущностью и любой сущностью-категорией устанавливается идентифицирующая зависимость типа «один к одному»: любому экземпляру сущности-категории обязательно соответствует один экземпляр обобщающей сущности. Кардинальность такой связи относится к типу «ноль или один» и обозначается меткой *Z*.

Сущность-категория может иметь только одну общую сущность, но она может быть обобщающей сущностью в другом отношении категоризации.

Категориальные связи бывают полные и неполные.

Связь называется **полной**, если любому экземпляру обобщающей сущности обязательно соответствует один экземпляр сущности-категории.

Если экземпляру обобщающей сущности нет соответствия среди экземпляров любой сущности-категории, то связь называется **неполной**.

Приведенный пример представляет полную категориальную связь.

Общие атрибуты-характеристики родовой сущности наследуются сущностями-категориями. Например, персональные характеристики сущности

«Сотрудник» по определению обобщения присущи и категориальным сущностям «Постоянный», «Совместитель», «Консультант».

Среди атрибутов обобщающей сущности должен быть определен атрибут-дискриминатор («тип»), который принимает значение имени категории и показывает, с какой из возможных сущностей-категорий устанавливается связь.

Каждый экземпляр обобщающей сущности и связанный с ним экземпляр одной из сущностей-категорий представляют один и тот же предмет реального мира и поэтому должны обладать одним и тем же уникальным идентификатором. Поэтому атрибуты первичного ключа сущности-категории должны совпадать с атрибутами первичного ключа обобщающей сущности, доставшимися сущности-категории «по наследству» идентифицирующей зависимости.

Еще один пример типа связи. Допустим, что в проектной организации любой сотрудник может быть занят в нескольких проектах, а в каждом проекте может быть занято несколько сотрудников. Причем могут быть сотрудники, не участвующие в разработке проектов, но не может быть проектов, никем не разрабатываемых. В этом случае между сущностями «Сотрудник» и «Проект» устанавливается бинарное отношение (связь) типа **«многие ко многим»** (рис. 8). Имя связи задается в обоих направлениях.

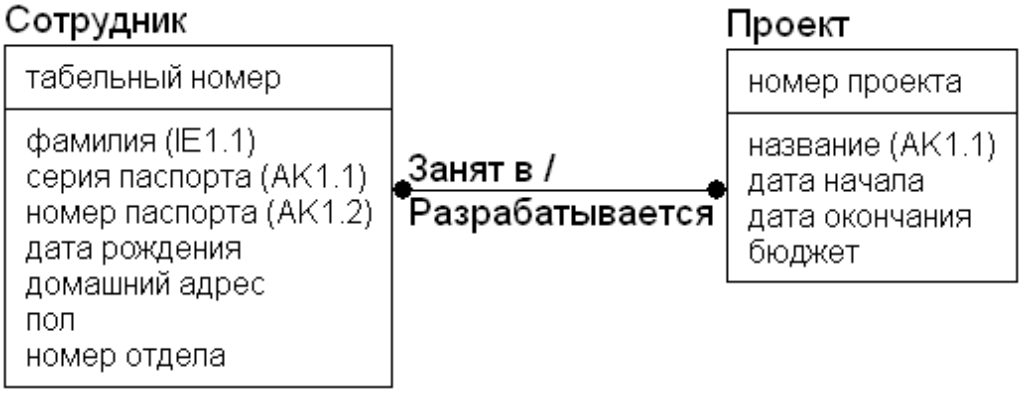


Рис. 8. Связь «многие ко многим»

Связи «многие ко многим» в общем виде устанавливаются на ER-диаграмме при первоначальной разработке модели. Впоследствии они

детализируются с целью отобразить существенные свойства связей, которые не могут быть представлены на обобщенной модели.

В процессе детализации связь «многие ко многим» моделируется дополнительной, т. н. **ассоциативной** сущностью, например в рассматриваемом случае «Проектирование», которая связана с основными сущностями «Сотрудник» и «Проект» как с родительскими сущностями идентифицирующими связями (рис. 9).

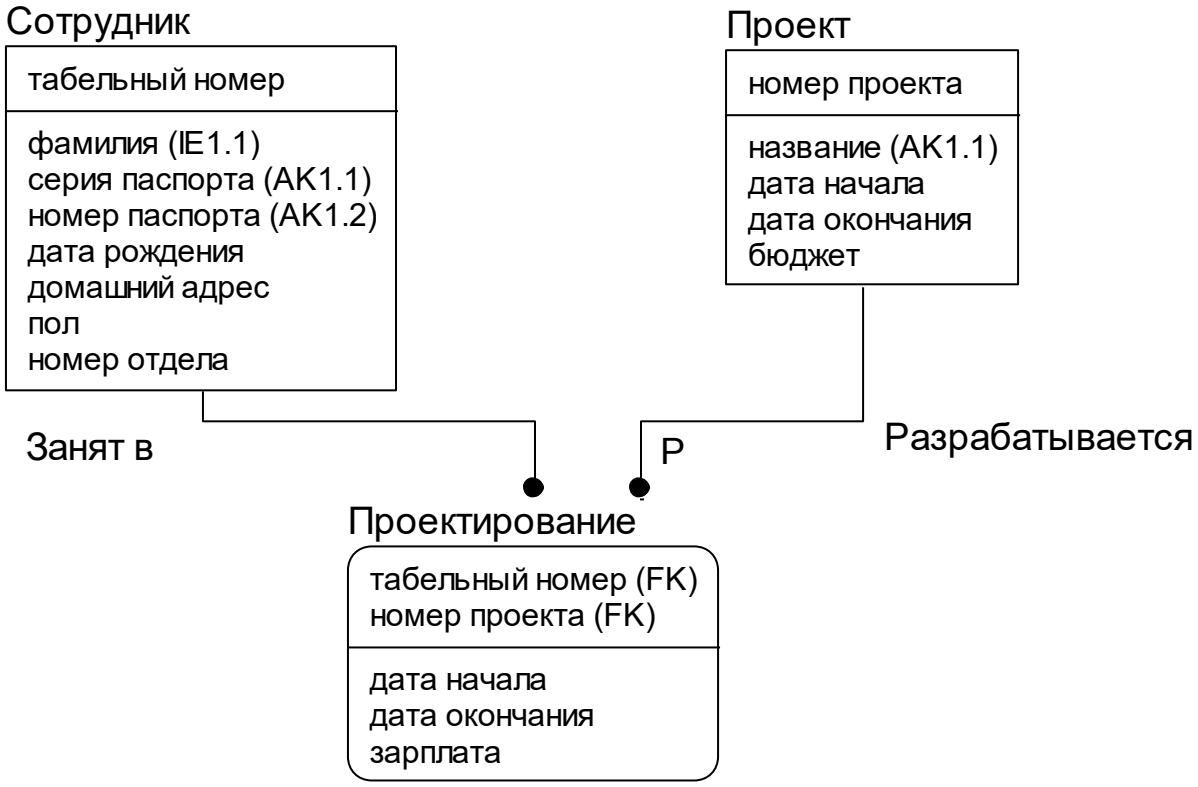


Рис. 9. Детализированная связь «многие ко многим»

Ассоциативная сущность кроме ключей связи с базовыми сущностями содержит свои собственные атрибуты – спецификаторы свойств связи «многие ко многим».

Ассоциативная сущность «Проектирование» показывает, в каких проектах и в какое время были заняты сотрудники, а также какие сотрудники принимали участие в разработке конкретных проектов.

Кардинальность идентифицирующих связей показывает, что не все сотрудники заняты в проектах (кардинальность связи «Занят в» относится к типу «ноль, один или много»), но нет проектов, над которыми никто не

работал (кардинальность связи «Разрабатывается» относится к типу *P* «один или много»).

Принцип моделирования бинарных связей «многие ко многим» может быть применен для моделирования N-арных связей.

Допустим, сотрудники некоторой организации когда-то получили какое-то образование и владеют или не владеют иностранными языками. Предлагаемая модельная ситуация регулируется следующими семантическими условиями:

– любой сотрудник мог закончить несколько учебных заведений или владеть несколькими иностранными языками;

– учебное заведение дает образование многим учащимся и может обучать многим иностранным языкам;

– иностранные языки преподают в разных учебных заведениях и изучают многие учащиеся;

– не может быть сотрудников, которые нигде не учились и не изучали иностранных языков;

– нет учебных заведений, которые никто не окончил, и где не изучают иностранных языков;

– могут быть иностранные языки, которые не преподают в учебных заведениях или не известны сотрудникам.

Ассоциативная сущность «Образование» моделирует тернарную связь (рис. 10) и показывает, какие сотрудники что окончили и владеют или нет иностранными языками.

Кардинальность идентифицирующих связей «Получил», «Дает» и «Дополняет» соответствует семантическим условиям.

Унарная (рекурсивная) связь – это связь сущности самой с собой. В результате одна и та же сущность является и потомком, и родителем по отношению к себе самой. Наиболее распространенным типом рекурсии является иерархическая рекурсия, характерная для структур управления.

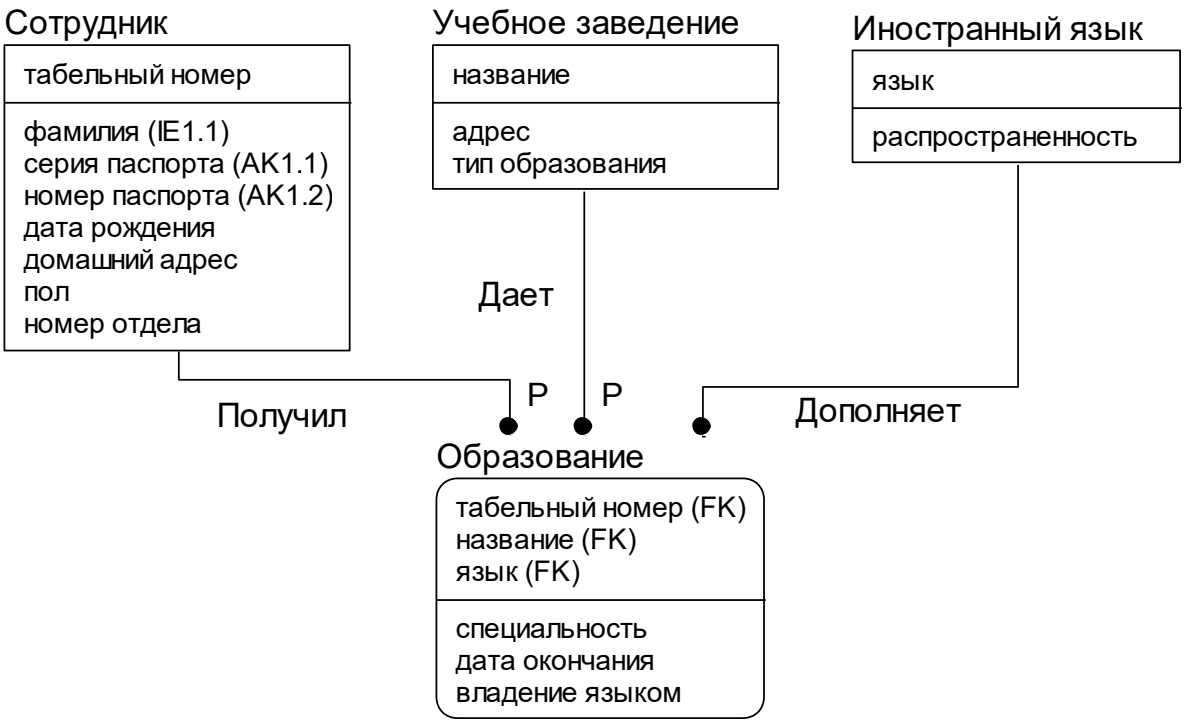


Рис. 10. Тернарная связь

Допустим, что в некотором контексте сущность «Сотрудник» должна представлять персональные характеристики не только самого сотрудника, но и его непосредственного начальника. Поскольку начальник является сотрудником той же организации, то для ссылки на него следует установить рекурсивную связь «Руководит/Подчиняется» (рис. 11).

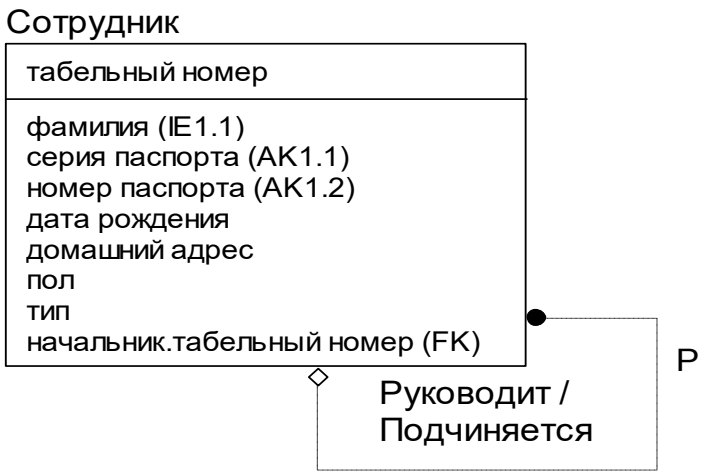


Рис. 11. Унарная рекурсивная связь

При задании рекурсивной связи первичный ключ («табельный номер») должен мигрировать в качестве внешнего ключа в состав неключевых атрибутов той же сущности. Поскольку один и тот же атрибут не может

появиться дважды под одним и тем же именем, внешнему ключу должно быть присвоено имя **роли** (например, «начальник»)

Рекурсивная связь может быть только неидентифицирующей. Иначе внешний ключ должен будет войти в состав первичного ключа и не сможет принимать неопределенное значение (Null). А это значит, что нельзя будет учесть самого старшего начальника, который никому не подчиняется в рамках данной организации.

4.4. Ограничения ссылочной целостности

Ограничения ссылочной целостности – это свойства, это правила, которые ограничивают выполнение корректирующих операций вставки (**Insert**)**,** обновления (**Update**) и удаления (**Delete**) экземпляров сущностей, одна из которых является родителем (**Parent**), а другая – потомком (**Child**).

Обеспечение ссылочной целостности требует, чтобы в модели данных не было не согласованных значений внешних ключей.

Операции, которые могут привести к нарушению ссылочной целостности:

- обновление первичного ключа экземпляра родительской сущности;
- удаление экземпляра родительской сущности;
- вставка экземпляра сущности-потомка;
- обновление внешнего ключа экземпляра сущности-потомка.

Выбор адекватного способа поддержания целостности при выполнении операций манипулирования данными определяется семантикой связей, бизнес-правилами предметной области.

Возможны следующие стратегии обеспечения ссылочной целостности:

– **RESTRICT** (запретить) – не разрешать выполнение операции, приводящей к нарушению ссылочной целостности.

– **CASCADE** (каскадировать) – разрешить выполнение требуемой операции, но внести при этом необходимые поправки в других сущностях так, чтобы не допустить нарушения ссылочной целостности и сохранить все имеющиеся связи.

– **SET NULL** (установить в Null) – разрешить выполнение требуемой операции, но все возникающие некорректные значения внешних ключей изменять на неопределенные (Null) значения.

– **SET DEFAULT** (установить по умолчанию) – разрешить выполнение требуемой операции, но все возникающие некорректные значения внешних ключей изменять на некоторое значение, принятое по умолчанию.

– **NO ACTION** (игнорировать) – выполнять операции, не обращая внимания на нарушения ссылочной целостности.

– **NONE** – не требуется обеспечение ссылочной целостности.

Выбор той или иной стратегии контроля ссылочной целостности определяется:

1) семантическими условиями, бизнес-правилами, действующими в моделируемой предметной области;
2) типом связи:
   – идентифицирующая;
   – неидентифицирующая (Null-значения внешнего ключа разрешены);
   – неидентифицирующая (Null-значения внешнего ключа запрещены);
   – категориальная;
3) типом сущности:
   – старшая (Parent) – родительская или обобщающая;
   – младшая (Child) – потомок или категория;
4) типом операции:
   – вставка (Insert);
   – удаление (Delete);
   – обновление (Update).

Возможные ограничения ссылочной целостности приведены в таблица 1 [9]. Значения по умолчанию выделены жирным шрифтом.

Аббревиатуры возможных реакций на выполнение операций вставки (I – Insert), удаления (D – Delete) и обновления (U – Update) соответствуют: R – Restrict, C – Cascade, SN – Set Null, SD – Set Default, None – No Action

Если в предметной области ограничения ссылочной целостности отличаются от умолчания, они могут быть заданы в явном виде при настройке свойств связей.

Таблица 1

Возможные ограничения ссылочной целостности

Тип сущности	**Тип операции**	**Тип связи**			
		Идентифицирующая	**Неидентифицирующая (Null разрешены)**	**Неидентифицирующая (Null запрещены)**	**Категориальная**
Parent	Insert	Restrict (IR), Cascade (IC), **None**	Restrict (IR), Cascade (IC), **None,** Set Null (ISN), Set Default (ISD)	Restrict (IR), Cascade (IC), **None,** Set Default (ISD)	Restrict (IR), Cascade (IC), **None**
	Delete	**Restrict (DR),** Cascade (DC), None	Restrict (DR), Cascade (DC), None, **Set Null (DSN),** Set Default (DSD)	**Restrict (DR)**, Cascade (DC), None, Set Default (DSD)	Restrict (DR), **Cascade (DC)**, None
	Update	**Restrict (UR),** Cascade (UC), None	Restrict (UR), Cascade (UC), None, **Set Null (USN)**, Set Default (USD)	**Restrict (UR),** Cascade (UC), None, Set Default (USD)	Restrict (UR), **Cascade (UC)**, None
Child	Insert	**Restrict (IR),** Cascade (IC), None	Restrict (IR), Cascade (IC), None, **Set Null (ISN)**, Set Default (ISD)	**Restrict (IR)**, Cascade (IC), None, Set Default (ISD)	**Restrict (IR)**, Cascade (IC), None
	Delete	Restrict (DR), Cascade (DC), **None**	Restrict (DR), Cascade (DC), **None**, Set Null (DSN)**,** Set Default (DSD)	Restrict (DR), Cascade (DC), **None**, Set Default (DSD)	Restrict (DR), Cascade (DC), **None**
	Update	**Restrict (UR),** Cascade (UC), None	Restrict (UR), Cascade (UC), None, **Set Null (**USN**)**, Set Default (USD)	**Restrict (UR)**, Cascade (UC), None, Set Default (USD)	**Restrict (UR)**, Cascade (UC), None

Допустим, объектом моделирования является некая организация, состоящая из отделов, в которых работают сотрудники. В предметной области действуют следующие бизнес-правила:

– возможно создание отделов, временно не укомплектованных сотрудниками;

– сотрудники, работающие в организации, могут быть временно не приписаны ни к одному отделу (например, при приеме на работу с испытательным сроком или при расформировании отдела);

– при изменении номера отдела соответствующие обновления должны быть автоматически произведены в свойствах сотрудников этого отдела;

любой сотрудник может быть уволен.В этом случае между сущностями «Отдел» и «Сотрудник» устанавливается неидентифицирующая связь (Null разрешены) под контролем помеченных ограничений целостности (рис. 12).



Рис. 12. Ограничения ссылочной целостности

Помеченные ограничения целостности имеют следующую интерпретацию.

Для родительской сущности «Отдел»:

I: NONE (не показано) – возможно создание отделов, временно не укомплектованных сотрудниками;

D:SN – при расформировании отдела сотрудники, работающие в организации, могут быть временно не приписаны ни к одному отделу;

U:C – изменение номера отдела должно быть передано по каскаду сущности-потомку для изменения ее внешнего ключа.

Для сущности-потомка «Сотрудник»:

I:SN – при приеме на работу сотрудники, работающие в организации, могут быть временно не приписаны ни к одному отделу;

U:SN – при переводе сотрудники, работающие в организации, могут быть временно не приписаны ни к одному отделу;

D:NONE (не показано) – любой сотрудник может быть уволен.

5. ПОСТРОЕНИЕ ИНФОРМАЦИОННОЙ МОДЕЛИ

Для закрепления навыков построения информационной модели в качестве учебного примера предметной области рассмотрим упрощенную информационную систему учета кадров на предприятии.

В качестве инструментальной среды для построения информационной модели используем популярный пакет ERwin [9,10]. Освоение ERwin не является целью данного учебного пособия. Главная задача состоит в освоении методологии информационного моделирования IDEF1X. ERwin в нашем случае используется лишь как средство реализации принципов построения информационной модели. Однако возможности ERwin гораздо богаче.

ERwin является популярным CASE-средством поддержки методологии IDEF1X. ERwin использует графические (диаграммные) средства для представления моделей данных, но ERwin не является просто графическим редактором. Основной целью использования ERwin является помощь аналитику в использовании бизнес-правил и требований к информации при создании логических и физических моделей данных.

Подробное описание технологии работы в среде ERwin можно найти в печатных изданиях [9,10] или электронных публикациях [11–13].

Построение информационной модели в ERwin предполагает выполнение предопределенных действий в заданной последовательности:

1. Создание логической ER-модели.
2. Определение физической модели данных.
3. Генерация схемы базы данных в среде целевой СУБД.

Методология моделирования сущностей и связей используется при создании логической модели предметной области. Поэтому ограничимся только этой стадией разработки информационной модели.

На стадии логического проектирования построение информационной модели предполагает выполнение следующих действий:

1. Создание обобщенной логической ER-модели.
    1.1. Установление потенциальных сущностей.
    1.2. Установление потенциальных связей.
    1.3. Определение сущностей и связей.
    1.4. Анализ обобщенной ER-модели.
2. Детализация ER-модели.
    2.1. Определение списка атрибутов для каждой сущности.
    2.2. Определение первичных и альтернативных ключей для каждой сущности.
    2.3. Описание всех атрибутов.
    2.4. Нормализация модели данных до требуемого уровня нормальной формы.
    2.5. Анализ детализированной логической модели.
3. Идентификация и учет бизнес правил.
    3.1. Анализ и определение кардинальности связей.
    3.2. Определение ограничений целостности.
    3.3. Определение доменов.
    3.4. Определение значений по умолчанию для атрибутов.

Создание новой модели в ERwin начинается по команде **File|New**. После создания модели рекомендуется по команде **Model|Model Properties** настроить ее свойства в диалоговом окне **Model Properties** (рис. 13).

1. На закладке **General** модели присваивается имя, например, «Кадры», можно указать свое авторство.
2. На закладке **Definition** следует дать содержательное описание модели, например, «Информационная система учета кадрового состава предприятия».
3. На закладке **Notation** следует настроиться на нотацию моделирования IDEF1X.

ERwin имеет контекстно-зависимые панели инструментов, «включаемые» в пункте **View** главного меню. Основные элементы размещаются на палитре инструментов (рис. 14).

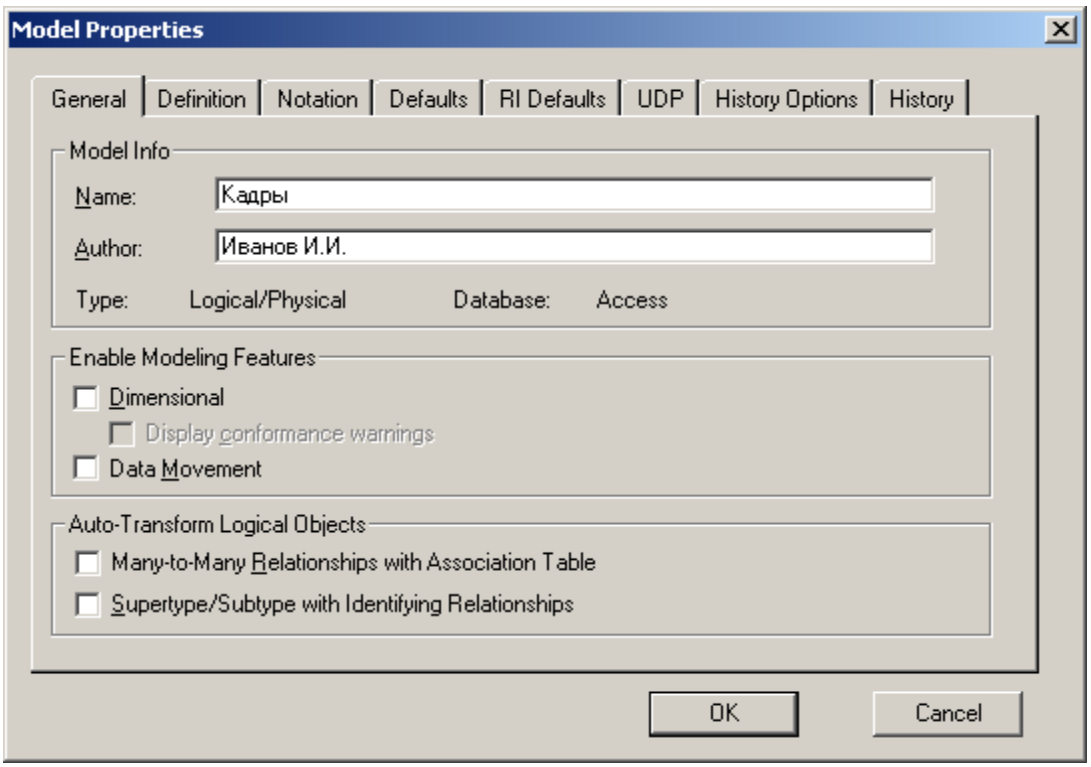


Рис. 13. Настройка свойств модели

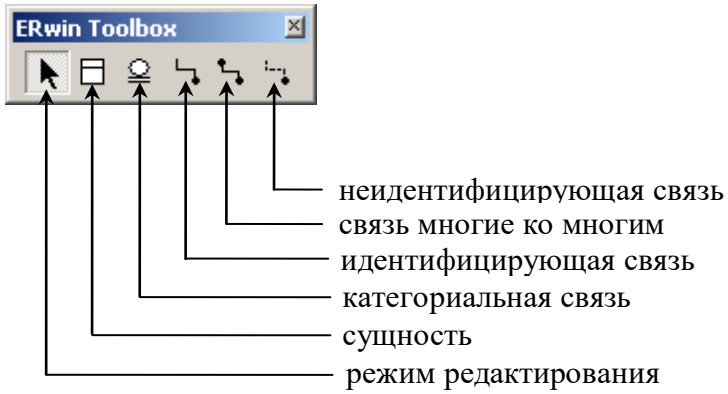


Рис. 14. Палитра инструментов ERwin Toolbox

5.1. Создание сущностей

Основанием для установления потенциальных сущностей моделируемой предметной области является анализ бизнес-процессов, происходящих в ней.

Допустим в нашей предметной области для описания бизнес-процессов в упрощенном виде достаточно следующих объектов (сущностей): «Сотрудник», «Отдел», категории сотрудников «Постоянный», «Совместитель», «Консультант», «Работа» (как место работы), «Учебное заведение», «Дети» (дети сотрудников).

Обобщенная логическая ER-модель имеет вид диаграммы сущность – связь **(Entity Relationship Diagram (ERD))** и представляет собой модель данных верхнего уровня. Она включает сущности и взаимосвязи, отражающие основные бизнес-правила предметной области.

Для внесения сущности в модель необходимо убедиться, что вы находитесь на уровне логической модели (**Logical**), и «кликнуть» по кнопке сущности  на панели инструментов **(ERwin Toolbox)**, затем «кликнуть» по тому месту на диаграмме, где необходимо расположить сущность (рис. 15).

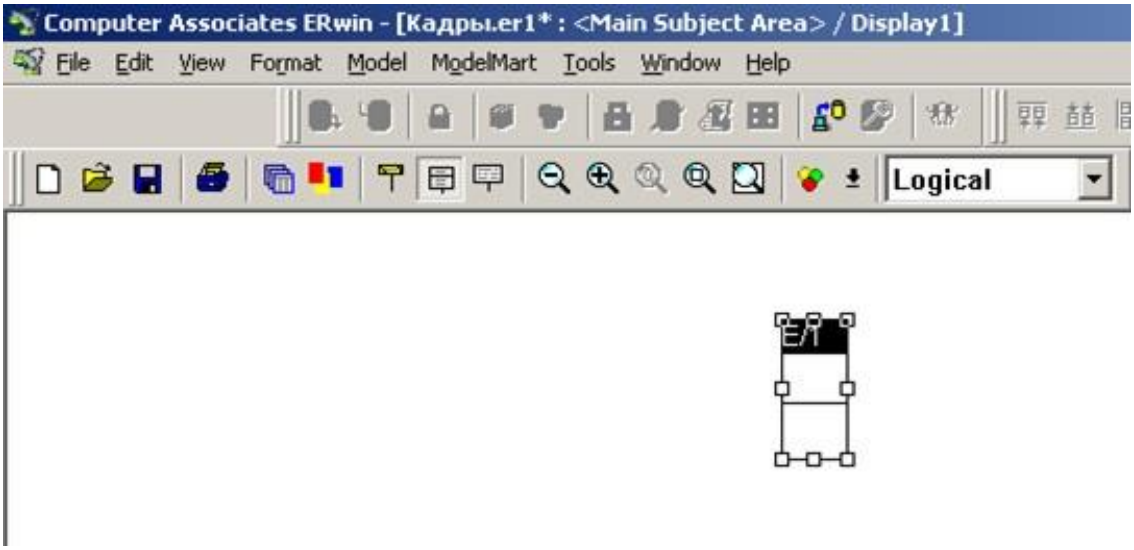


Рис. 15. Создание сущности

Для спецификации свойств сущности следует щелкнуть правой кнопкой мыши по ее шаблону и выбрать в контекстном меню пункт **Entity Properties.**

В диалоге **Entities** на закладке **Definition** следует присвоить имя сущности в окне **Name** и рекомендуется дать ее содержательное описание в окне **Definition** (рис. 16).

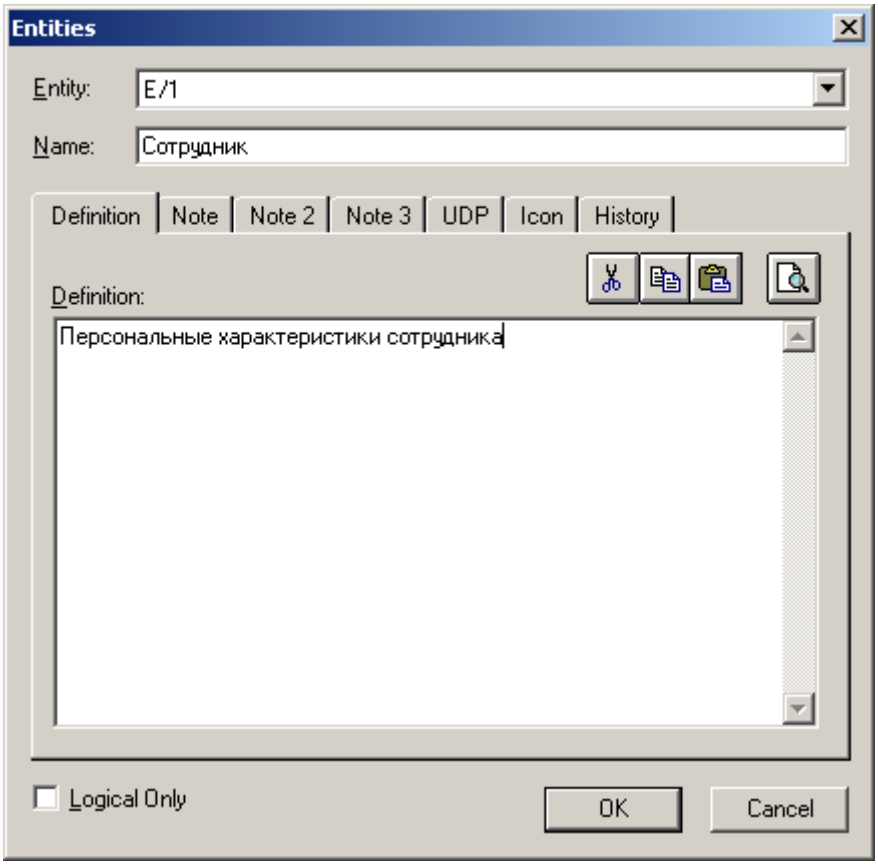


Рис. 16. Диалог спецификации свойств сущности

По образцу можно создать и определить свойства всех остальных сущностей. При определении свойств сущностей можно руководствоваться их спецификациями, приведенными в таблица 2. Блоки сущностей можно расположить примерно так, как показано на рис. 17. Перемещение блоков сущностей выполняется мышью без проблем.

Таблица 2

Свойства сущностей

Имя	Описание
Дети	Дети сотрудника
Консультант	Сотрудник, временно привлекаемый "со стороны" для консультации
Отдел	Место работы сотрудника
Постоянный	Постоянный, штатный сотрудник
Работа	Места работы и должности сотрудника на протяжении его трудовой деятельности (послужной список)
Совместитель	Сотрудник, совмещающий работу на предприятии с основной работой в другом месте
Сотрудник	Персональные характеристики сотрудника
Учебное заведение	Учебные заведения, которые окончил сотрудник

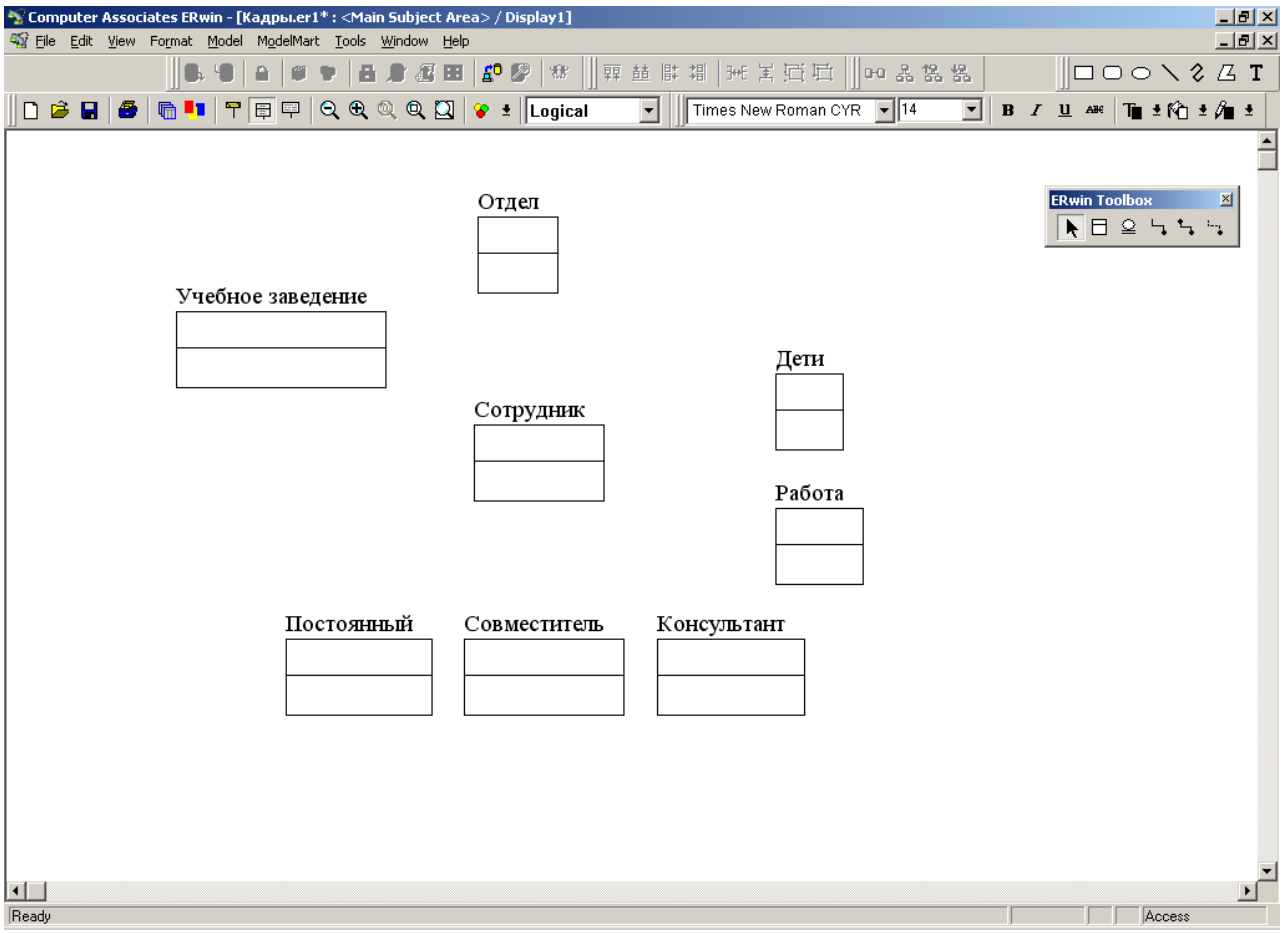


Рис. 17. Сущности моделируемой предметной области

5.2. Создание связей

Тип связи определяется бизнес-правилами, семантическими условиями, действующими в предметной области. Допустим, в нашей организации, которая состоит из отделов, возможно создание отделов, временно не укомплектованных сотрудниками. Допустим также, что сотрудники, работающие в организации, могут быть временно не приписаны ни к одному отделу. В этом случае между сущностями «Отдел» и «Сотрудник» имеется неидентифицирующая связь, потому что персона сотрудника идентифицируется своими средствами (например, табельным номером), а не связью с отделом.

Для внесения неидентифицирующей связи между сущностями «Отдел» и «Сотрудник» необходимо «кликнуть» по кнопке неидентифицирующей связи  на панели инструментов **(ERwin Toolbox)**, затем «кликнуть» по родительской сущности («Отдел») и по сущности-потомку («Сотрудник»). Направление связи имеет принципиальное значение.

Для спецификации свойств связи следует щелкнуть правой кнопкой мыши по ее шаблону и выбирать в контекстном меню пункт **Relationship Properties.**

В диалоге **Relationships** на закладке **General** следует присвоить связи имя, кардинальность, тип (рис. 18).

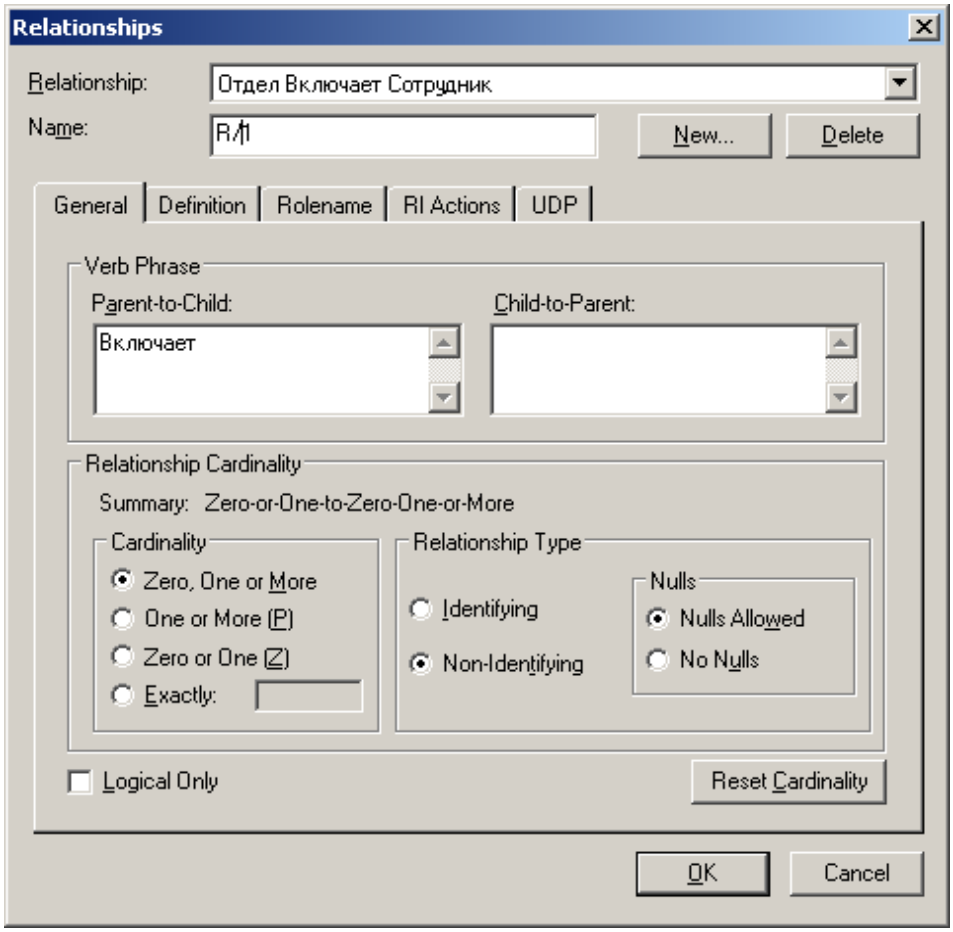


Рис. 18. Диалог спецификации свойств связи

Имя связи (Verb Phrase) – фраза, характеризующая отношение между родительской сущностью и сущностью-потомком. Для связи «один ко многим» идентифицирующей или неидентифицирующей достаточно указать имя, характеризующее отношение родительской сущности к сущности-потомку **(Parent-to-Child)**. Для связи «многие ко многим» следует указывать имена как **Parent-to-Child** так и **Child-to-Parent**. Связь между сущностями «Отдел» и «Сотрудник» можно назвать, например, «Включает».

Мощность связи (Cardinality) – служит для обозначения отношения одного экземпляра родительской сущности к числу экземпляров сущности-потомка. Мощность (кардинальность) связи бывает четырех типов:

– ноль, один или много (не помечается);

– один или много (помечается символом P (positive));

– ноль или один (помечается символом Z (zero);

– конкретное число экземпляров (помечается целым числом).

В случае связи «Включает» кардинальность будет ноль, один или много.

Тип связи (Identifying/Non-Identifying). Для неидентифицирующей связи можно указать обязательность (**No Nulls**) или необязательность (**Nulls Allowed**) для ключа связи сущностей (внешнего ключа) принимать значение «не определено» (Null). В случае связи «Включает» экземпляры сущности «Сотрудник» могут существовать без связи с родительской сущностью «Отдел», поэтому обязательность связи имеет значение по умолчанию для неидентифицирующей связи **Nulls Allowed**.

На закладке **Definition** диалога **Relationships** рекомендуется дать содержательное описание связи, которое является полезной метаинформацией. Например, связь «Включает» можно описать так «Отдел может включать в свой состав сотрудника, но сотрудник может быть не приписан ни к одному отделу».

В закладке **Rolename** можно задать имя роли внешнего ключа.

В закладке **RI Actions** правила ссылочной целостности.

По умолчанию имя и другие свойства связи на ER-диаграмме не показывается. Для отображения свойств связи следует в контекстном меню, которое появляется, если щелкнуть правой кнопкой мыши по любому месту диаграммы, не занятому объектами модели, выбрать пункт **Relationship Display** и затем «включить» опции **Verb Phrase** и **Cardinality** (рис. 19).

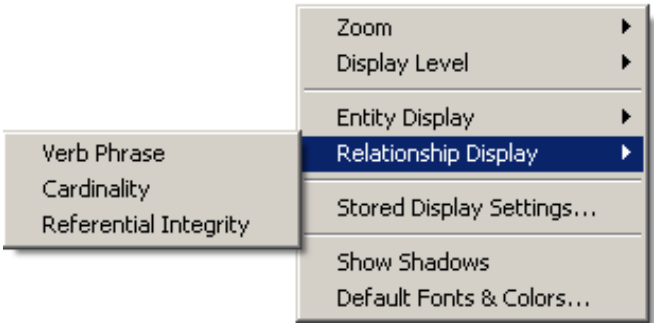


Рис. 19. Контекстное меню настройки отображения свойств связей

Между сущностями «Сотрудник», с одной стороны, «Работа» и «Дети», с другой, имеются идентифицирующие связи.

Персона конкретного ребенка в контексте учета кадров на предприятии не может идентифицироваться вне связи с его родителем – конкретным сотрудником. Точно также этапы послужного списка не могут идентифицироваться вне связи с сотрудником, чью трудовую деятельность они представляют.

Для внесения идентифицирующей связи между сущностями «Сотрудник» и «Дети», а также «Сотрудник» и «Работа» необходимо «кликнуть» по кнопке идентифицирующей связи  на панели инструментов **(ERwin Toolbox)**, затем «кликнуть» по родительской сущности («Сотрудник») и по сущности-потомку («Дети» или «Работа»). Направление связи имеет принципиальное значение.

Для спецификации свойств связи следует щелкнуть правой кнопкой мыши по ее шаблону и выбрать в контекстном меню пункт **Relationship Properties.**

В диалоге **Relationships** на закладках **General** и **Definition** следует присвоить имя связи, кардинальность, тип связи и описание, руководствуясь таблица 3.

Таблица 3

Свойства связей сущности «Сотрудник» с сущностями «Дети» и «Работа»

Родитель	**Потомок**	**Имя связи (Verb Phrase)**	**Кардинальность (Cardinality)**	**Описание (Definition)**
Сотрудник	Дети	Имеет	Ноль, один или много	Сотрудник может не иметь детей или иметь одного ребенка и более
Сотрудник	Работа	Сменил	Один или много (P)	У сотрудника обязательно есть хотя бы одно место работы в настоящем или более в прошлом

Между сущностями «Сотрудник» и «Учебное заведение» имеется бинарная связь «многие ко многим». Это так потому, что любое учебное заведение может выпустить нескольких сотрудников данного предприятия. В то же время любой сотрудник может окончить несколько учебных заведений.

Связи «многие ко многим» в общем виде устанавливаются на ER-диаграмме при первоначальной разработке модели. Впоследствии они детализируются с целью отобразить существенные свойства связей, которые не могут быть представлены на обобщенной модели.

Для внесения связи «многие ко многим» между сущностями «Сотрудник» и «Учебное заведение» необходимо «кликнуть» по кнопке  на панели инструментов **(ERwin Toolbox)**, затем «кликнуть» по сущности «Сотрудник» и по сущности «Учебное заведение». Направление связи «многие ко многим» не имеет значения.

Для спецификации свойств связи следует щелкнуть правой кнопкой мыши по ее шаблону и выбрать в контекстном меню пункт **Relationship Properties**. В диалоге **Relationships** на закладке **General** можно присвоить имя связи. Для связи «многие ко многим» следует указывать имена как **Parent-to-Child**, так **Child-to-Parent**. Можно присвоить, например, связи в направлении от «Учебное заведение» к «Сотрудник» имя «Дает образование», а в обратном направлении – «Получил образование». На закладке **Definition** следует дать описание связи, например, «Связь показывает, какие сотрудники какие окончили учебные заведения».

Допустим, сотрудники нашей организации могут работать в качестве постоянных (штатных) сотрудников, в качестве совместителей или в качестве внешних консультантов. У всех этих категорий сотрудников есть некоторые общие, личностные характеристики, которые могут послужить основанием для их обобщения и создания новой обобщающей сущности.

Для моделирования такого рода отношений между сущностями-категориями «Постоянный», «Совместитель», «Консультант» и обобщающей сущностью «Сотрудник» используется связь на основе иерархии обобщения, т. н. **категориальная** (родовидовая) связь.

Для создания категориальной связи следует:

1. Щелкнуть по кнопке  на панели инструментов **(ERwin Toolbox)**.

2. Щелкнуть сначала по родовой сущности («Сотрудник»), а затем по категориальной (например, «Постоянный»).

3. Для установления связи с другими категориями следует сначала щелкнуть по символу  , а затем по категориальной сущности.

Категориальные связи не именуются.

Для спецификации свойств категориальной связи следует щелкнуть правой кнопкой мыши по ее символу и выбрать в контекстном меню пункт **Subtype Relationship**. Для настройки свойств категориальной связи на обобщенной ER-модели недостаточно данных, поэтому отложим эти действия до построения детализированной ER-диаграммы.

В некоторых случаях семантика данных предметной области может обуславливать т.н. **унарные** (рекурсивные) связи. Допустим, что в некотором контексте сущность «Сотрудник» должна представлять персональные характеристики не только самого сотрудника, но и его непосредственного начальника. Поскольку начальник является сотрудником той же организации, то для ссылки на него следует установить рекурсивную связь.

Рекурсивная связь может быть только неидентифицирующей. Иначе нельзя будет учесть самого старшего начальника, который никому не подчиняется в рамках данной организации.

Для внесения рекурсивной связи, определенной на сущности «Сотрудник» необходимо «кликнуть» по кнопке неидентифицирующей связи  на панели инструментов **(ERwin Toolbox)**, затем «кликнуть» дважды по сущности, на которой она определена («Сотрудник»).

Для спецификации свойств рекурсивной связи следует щелкнуть правой кнопкой мыши по ее шаблону и выбирать в контекстном меню пункт **Relationship Properties.**

В диалоге **Relationships** на закладке **General** можно присвоить имя связи, кардинальность, тип связи.

Разные экземпляры сущности «Сотрудник», связанные рекурсивной связью, могут выступать в роли родителя или в роли потомка. Поэтому для

рекурсивной связи следует указывать имена как **Parent-to-Child** (например, «Руководит»), так и **Child-to-Parent** (например, «Подчиняется»).

Рекурсивная связь в нашем случае будет иметь значение один или много (тип P) потому, что не может быть начальника, у которого нет подчиненных.

Экземпляр сущности «Сотрудник», соответствующий самому старшему начальнику, может существовать без связи с родителем, поэтому обязательность связи имеет значение по умолчанию для неидентифицирующей связи **Nulls Allowed**.

В диалоге **Relationships** на закладке **Definition** следует дать описание рекурсивной связи, например, такое: «Один сотрудник руководит или подчиняется другому сотруднику».

После спецификации свойств сущностей и связей учебной предметной области ее обобщенная ER-модель будет иметь как на рис. 20.

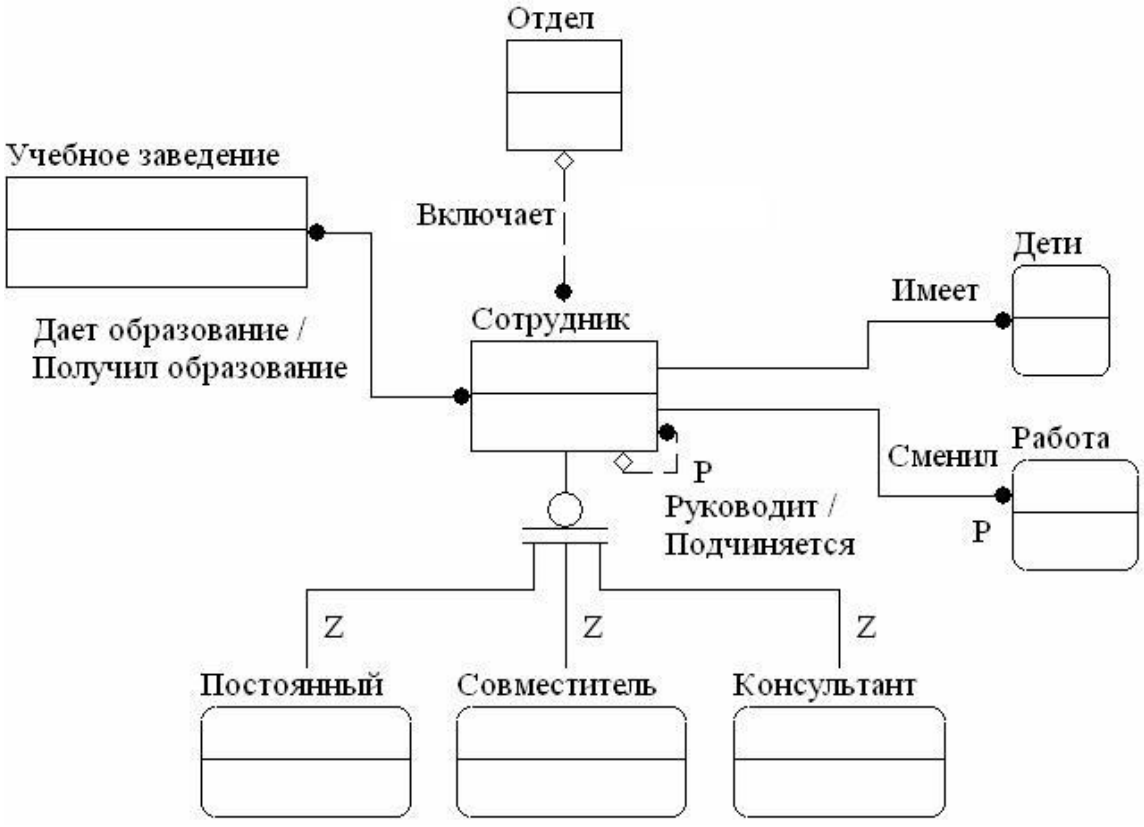


Рис. 20. Обобщенная ER-модель предметной области

5.3. Создание атрибутов

Детализированная ER-модель включает в себя спецификации не только сущностей и связей, но и атрибутов. Атрибуты раскрывают структуру сущностей и связей.

Цель детализации ER-модели состоит в построении полной атрибутивной модели предметной области (**Fully Attributed model (FA)**).

Рассмотрим в качестве примера сущность «Отдел» нашей учебной модели. Допустим, что структура сущности «Отдел» раскрывается при помощи атрибутов, представленных в таблица 4. Спецификация каждого атрибута предполагает задание имени атрибута, домена, на котором он определен, и факультативно указание особой роли атрибута в качестве первичного или альтернативного ключа или индекса.

Таблица 4

Атрибуты сущности «Отдел»

Имя	Домен	Первичный ключ	Альтернативный ключ	Неуникальный индекс
Номер отдела	String	Да	Нет	Нет
Название отдела	String	Нет	Да	Нет
Бюджет	Number	Нет	Нет	Нет

По умолчанию ERwin поддерживает четыре предопределенных домена:

Blob – большой бинарный объект, используется для представления в модели слабо структурированных данных.

Datetime – дата и время.

Number – число.

String – строка символов.

На основе предопределенных доменов могут быть созданы другие домены.

Для включения в ER-модель атрибутов сущности следует, «кликнув» правой кнопкой по сущности, выбрать в контекстном меню пункт **Attributes** (рис. 21).

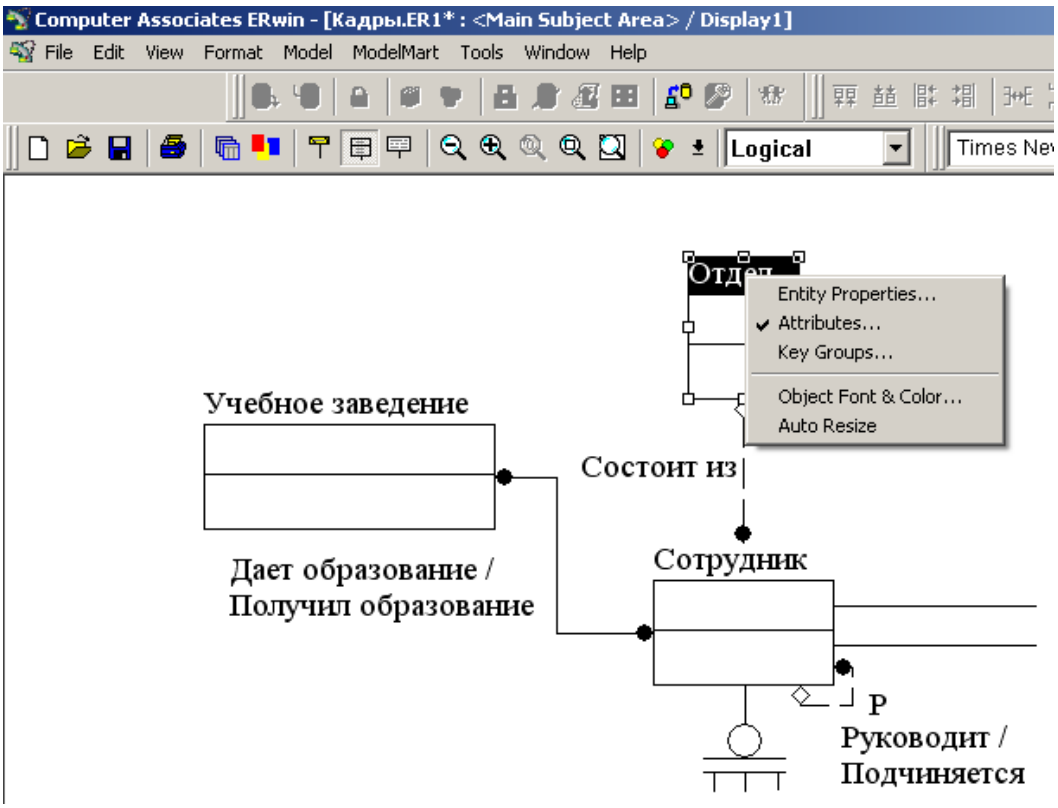


Рис. 21. Создание атрибутов

В диалоге **Attributes** (рис. 22) можно создать или отредактировать свойства атрибутов сущности.

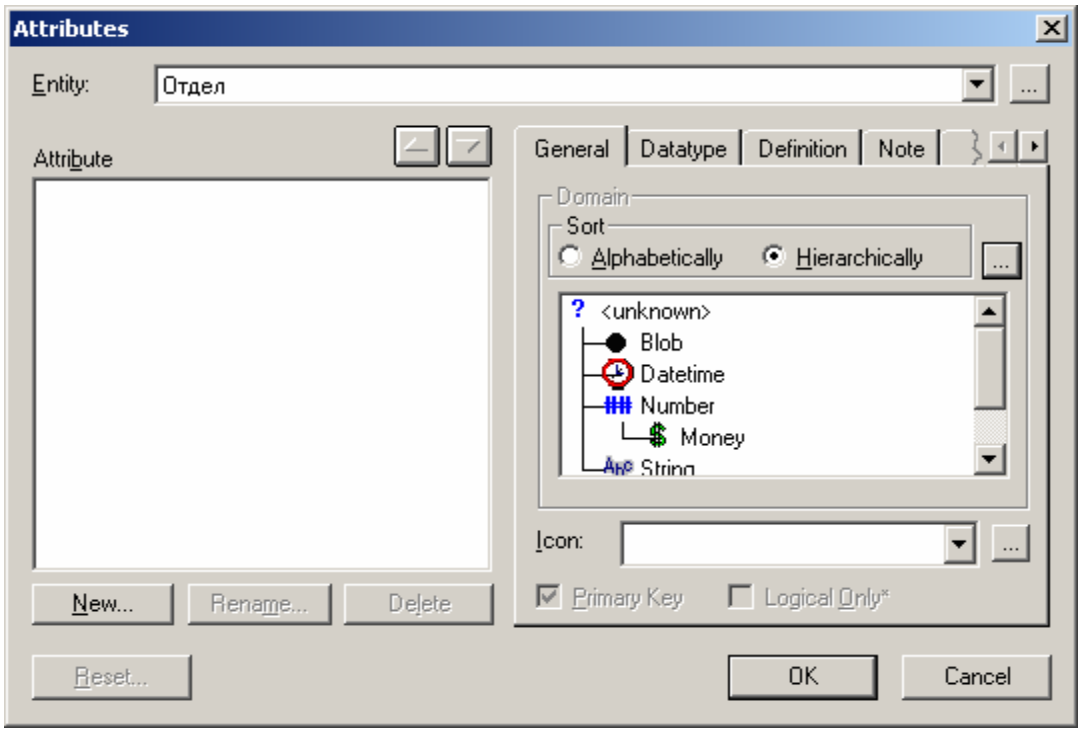


Рис. 22. Диалог спецификации свойств атрибутов

Любой атрибут можно создать (New), переименовать (Rename), удалить (Delete).

Если щелкнуть по кнопке **New…**, то в появившемся диалоге **New Attribute** можно указать имя атрибута, имя соответствующей ему в физической модели колонки и домен (рис. 23).

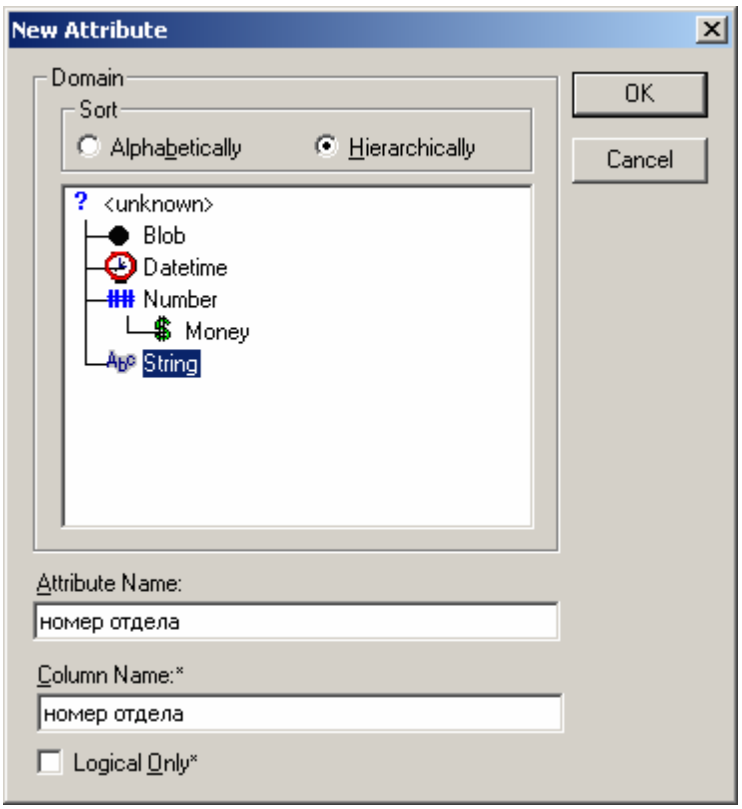


Рис. 23. Диалог создания атрибута

Атрибутам как спецификаторам свойств сущностей или связей присваиваются семантически значимые имена, как правило, в форме существительного, например, «номер отдела». Очень важно дать атрибуту правильное имя. Атрибуты должны именоваться в единственном числе и иметь четкое смысловое значение. Соблюдение этого правила позволяет частично решить проблему нормализации данных уже на этапе определения атрибутов. Согласно синтаксису IDEF1X имя атрибута должно быть уникально в рамках модели (а не только в рамках сущности!).

В диалоге **Attribute** на закладке **General** атрибуту можно присвоить статус первичного ключа (включить флажок **Primary Key**) (рис. 24) или сделать это на закладке **Key Group**

Закладка **Datatype** позволяет выбрать тип атрибута.

Закладка **Definition** позволяет сформулировать описание атрибута. Например, для атрибута «номер отдела» – «Уникальный номер отдела».

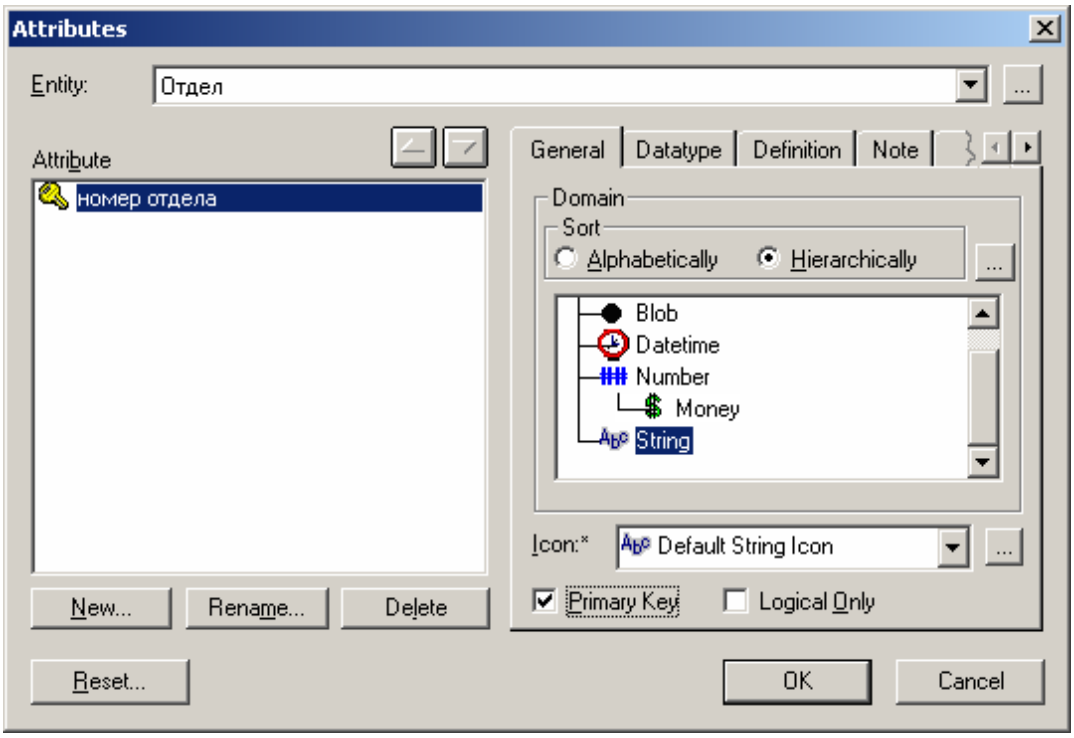


Рис. 24. Определение первичного ключа сущности

По аналогии с атрибутом «табельный номер» можно определить на домене **String** второй атрибут сущности «Отдел» «название отдела». Этот атрибут также является потенциальным ключом, но уступает по компактности «номеру отдела», поэтому он должен быть определен как альтернативный ключ. Атрибут «название отдела» можно сопроводить описанием на закладке свойств атрибутов **Definition**: «Уникальное название отдела»

Аналогично можно определить третий атрибут сущности «Отдел» «бюджет», который может быть определен на домене **Number**. В качестве описания этого атрибута можно предложить: «Годовой бюджет отдела».

5.4. Определение первичных и альтернативных ключей

Создать первичные и альтернативные ключи, а так же инверсионные входы (неуникальные индексы) можно на закладке **Key Group** диалога **Attributes** (рис. 25). Рассмотрим, как выполняется настройка свойств атрибутов в качестве альтернативного ключа на примере атрибута «название отдела» сущности «Отдел».

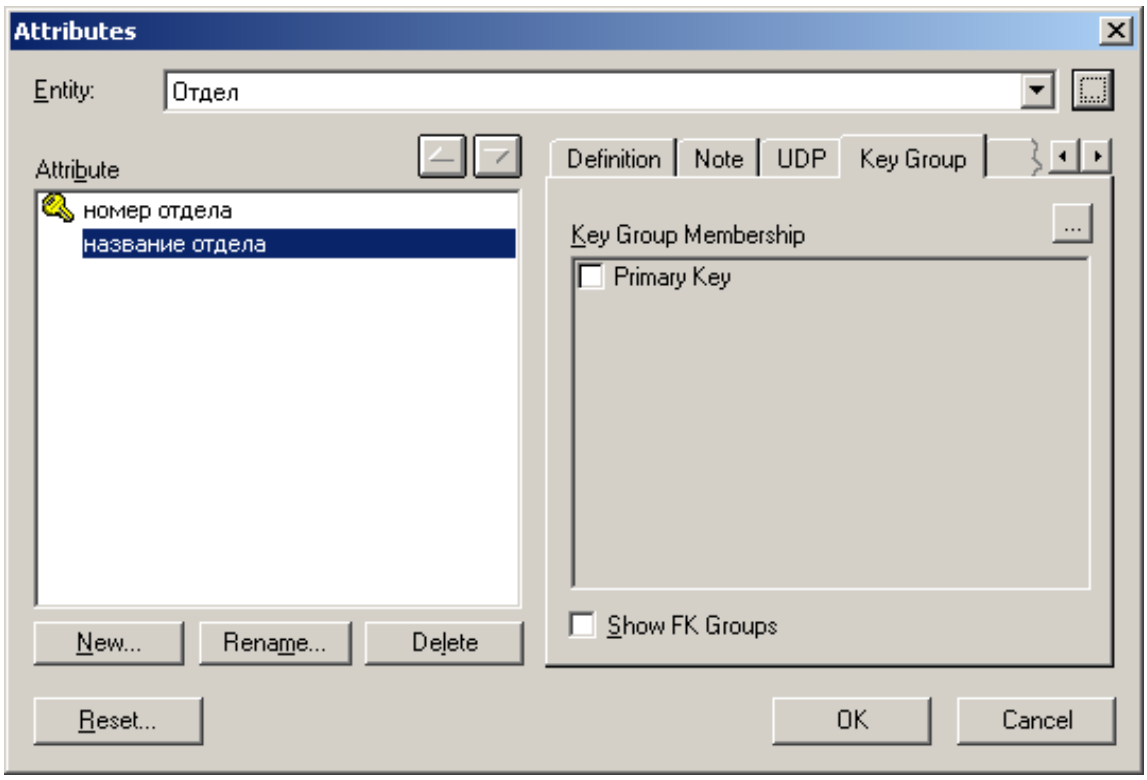


Рис. 25. Определение ключей и индексов сущности

Во-первых, необходимо выделить атрибут, подлежащий настройке. Если щелкнуть по кнопке ..., расположенной в правой верхней части закладки, вызывается диалог **Key Groups** (рис. 26). В верхней части диалога находится список ключей, в нижней – список атрибутов, доступных для включения в состав ключа (слева), и список ключевых атрибутов. Каждый вновь созданный ключ должен иметь хотя бы один атрибут. Для включения атрибута в состав ключа следует выделить его в левом списке и щелкнуть по кнопке со стрелкой .

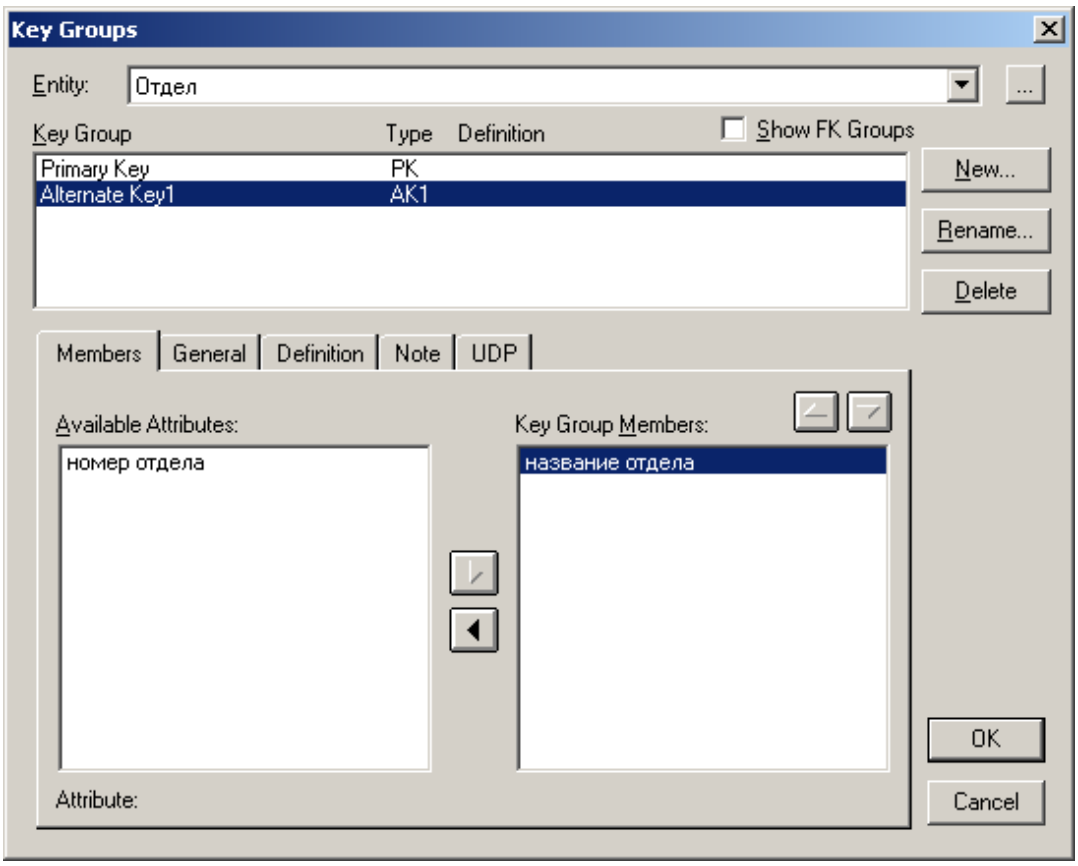


Рис. 26. Диалог определения ключей и индексов

Для создания нового ключа следует щелкнуть по кнопке **New…**. Появляется диалог **New Key Group** (рис. 27). Имя нового ключа присваивается автоматически (“Alternate Key N” для альтернативного ключа и “Inversion Entry N” для инверсионного входа, где N – порядковый номер ключа). На ER-диаграмме атрибуты альтернативного ключа будут помечены меткой **AKN.M**, а атрибуты инверсионного входа – меткой **IEN.M**, где N – номер ключа или входа, M – номер компонента составного ключа или входа.

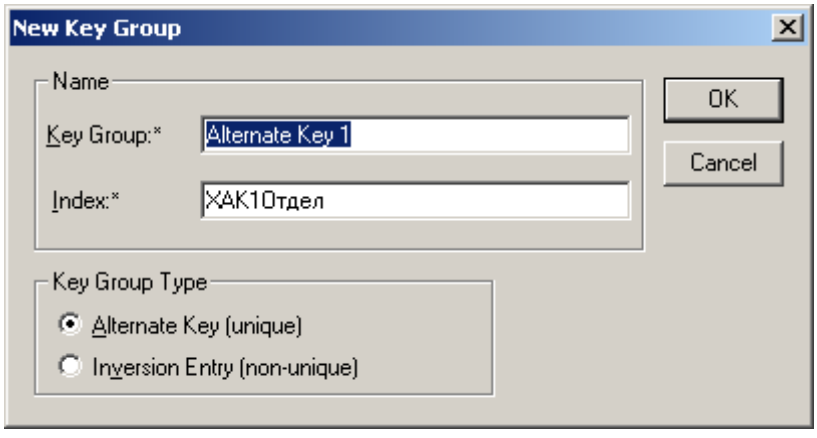


Рис. 27. Диалог создания альтернативного ключа и индекса

Для представления на ER-диаграмме максимума информации об атрибутах следует в контекстном меню, которое появляется, если щелкнуть правой кнопкой мыши по любому месту диаграммы, не занятому объектами модели, выбрать пункт **Entity Display** и затем «включить» необходимые (помеченные) опции (рис. 28).

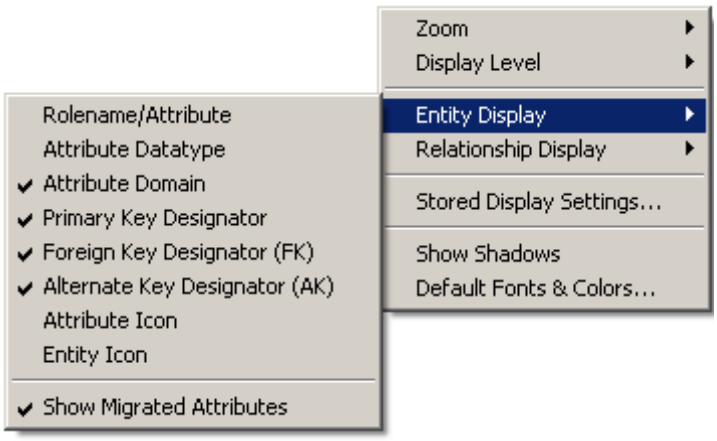


Рис. 28. Контекстное меню настройки отображения свойств сущностей

Обратите внимание на изменения ER-диаграммы (рис. 29). В структуре сущности «Отдел» появились ее атрибуты, причем первичный ключ «номер отдела» располагается над горизонтальной чертой. Кроме того, в структуре сущности «Сотрудник» появился «чужой» атрибут «номер отдела». Это первичный ключ родительской сущности «Отдел», мигрировавший в сущность-потомок и моделирующий неидентифицирующую связь типа «один ко многим» между сущностями «Отдел» и «Сотрудник». Это так называемый **внешний ключ (Foreign Key)** сущности «Сотрудник».

Внешний ключ появился в структуре сущности «Сотрудник» автоматически благодаря тому, связь между сущностями уже была ранее определена. Обратите внимание, что внешний ключ, моделирующий неидентифицирующую связь, размещается под горизонтальной чертой в области неключевых атрибутов, т. е. не входит в состав первичного ключа сущности «Сотрудник». Это специфика неидентифицирующих связей.

По завершении всех настроек в диалоге **Attributes** будут определены все атрибуты сущности «Отдел» (рис. 29).

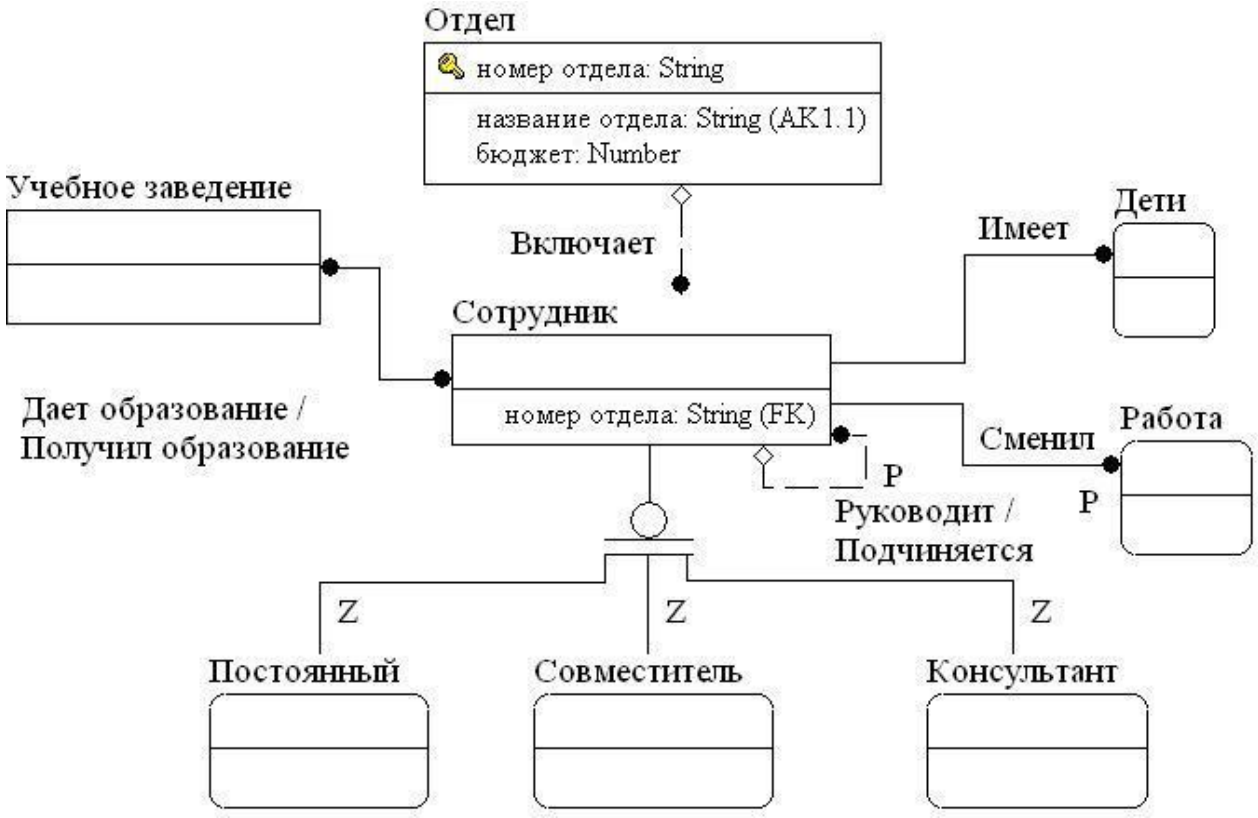


Рис. 29. Частично детализированная ER-модель

5.5. Детализация ER-модели

Основные приемы создания атрибутов мы рассмотрели, поэтому вы в состоянии выполнить эту работу самостоятельно. По примеру сущности «Отдел» создайте атрибуты других сущностей. Для не ключевого атрибута «фамилия» таблицы «Сотрудник» для примера создайте индекс. При определении свойств атрибутов можно руководствоваться их спецификациями, приведенными в таблица 5. Рекомендуется начинать с определения атрибутов старшей (родительской) сущности. Тогда в младших сущностях (потомках) их внешние ключи появятся автоматически.

Таблица 5

Спецификация свойств атрибутов

Имя сущности	Имя атрибута	Описание атрибута	Первичный ключ	Внешний ключ	Домен
Дети	табельный номер	Уникальный идентификатор личности сотрудника. Ссылка на родителя ребенка	Да	Да	String

Имя сущности	Имя атрибута	Описание атрибута	Первичный ключ	Внешний ключ	Домен
Дети	имя	Имя ребенка. Полагается, что в семье нет детей-тезок. Вместе с ссылкой на родителя является уникальной характеристикой ребенка	Да	Нет	String
Дети	дата рождения ребенка	Дата рождения ребенка	Нет	Нет	Datetime
Консультант	табельный номер	Уникальный идентификатор личности сотрудника	Да	Да	String
Консультант	организация	Место постоянной работы сотрудника	Нет	Нет	String
Отдел	номер отдела	Уникальный номер отдела	Да	Нет	String
Отдел	название отдела	Уникальное название отдела	Нет	Нет	String
Отдел	бюджет	Годовой бюджет отдела	Нет	Нет	Number
Постоянный	табельный номер	Уникальный идентификатор личности сотрудника	Да	Да	String
Постоянный	стаж	Стаж непрерывной работы на предприятии	Нет	Нет	Number
Работа	табельный номер	Уникальный идентификатор личности сотрудника. Ссылка на сотрудника – хозяина послужного списка	Да	Да	String
Работа	организация	Название организации, где работал сотрудник	Да	Нет	String
Работа	должность	Должность сотрудника по месту работы	Да	Нет	String
Работа	дата приема	Дата приема сотрудника на работу или перевода с одной должности на другую	Да	Нет	Datetime
Работа	дата увольнения	Дата увольнения сотрудника	Нет	Нет	Datetime
Работа	зарплата	Зарплата (оклад) сотрудника по месту его работы	Нет	Нет	Number
Совместитель	табельный номер	Уникальный идентификатор личности сотрудника	Да	Да	String
Совместитель	ставка	Доля ставки по данной должности	Нет	Нет	Number

Имя сущности	Имя атрибута	Описание атрибута	Первичный ключ	Внешний ключ	Домен
Сотрудник	табельный номер	Уникальный идентификатор личности сотрудника	Да	Нет	String
Сотрудник	начальник	Уникальный идентификатор личности сотрудника	Нет	Да	String
Сотрудник	номер отдела	Место работы сотрудника	Нет	Да	String
Сотрудник	фамилия	Фамилия и инициалы сотрудника	Нет	Нет	String
Сотрудник	дата рождения	Дата рождения сотрудника	Нет	Нет	Datetime
Сотрудник	домашний адрес	Адрес сотрудника (улица, дом, квартира) в виде строки символов	Нет	Нет	String
Сотрудник	серия паспорта	Серия паспорта сотрудника, вместе с номером паспорта является уникальной характеристикой личности сотрудника	Нет	Нет	String
Сотрудник	номер паспорта	Номер паспорта сотрудника, вместе с серией паспорта является уникальной характеристикой личности сотрудника	Нет	Нет	String
Сотрудник	пол	Пол сотрудника (мужской или женский)	Нет	Нет	String
Сотрудник	тип	Тип категории сотрудника: постоянный, совместитель, консультант	Нет	Нет	String
Учебное заведение	название	Уникальное название учебного заведения	Да	Нет	String
Учебное заведение	адрес	Адрес учебного заведения	Нет	Нет	String
Учебное заведение	тип образования	Тип образования, которое дает учебное заведение: среднее общее, среднее специальное, высшее	Нет	Нет	String

В окончательном виде полная атрибутивная модель должна принять вид, приведенный на диаграмме (рис. 30).

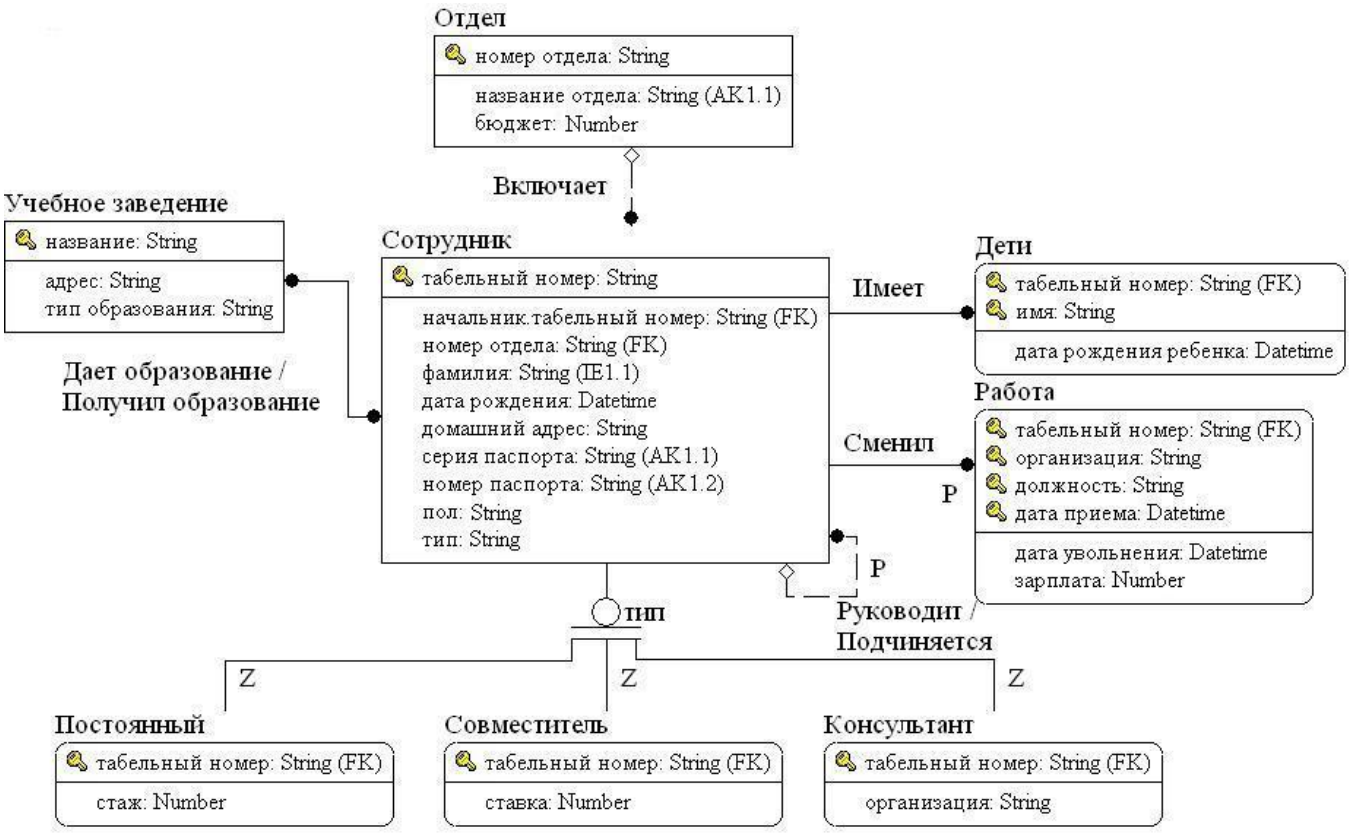


Рис. 30. Полная атрибутивная ER-модель

Отметим некоторые особенности структуры моделируемых сущностей после определения их атрибутов.

Сущности «Дети» и «Работа» имеют составные первичные ключи. Это обстоятельство обусловлено семантикой понятий «дети сотрудника» и «послужной список сотрудника».

В спецификации свойств сущности «Сотрудник» появился атрибут-**дискриминатор** «тип». Дискриминатор является неотъемлемой характеристикой категориальной связи, он показывает, как отличить одну категориальную сущность от другой.

Для редактирования категориальной связи нужно щелкнуть правой кнопкой мыши по символу  и выбрать в контекстном меню пункт **Subtype Relationship…** В диалоге **Subtype Relationship** (рис. 31) следует указать атрибут – дискриминатор категорий (список **Discriminator Attribute Choice**) и тип категории – полная или неполная (радиокнопки **Complete** и **Incomplete**).

После редактирования на диаграмме рядом с символом категориальной связи появится имя атрибута-дискриминатора «тип» (рис. 30).

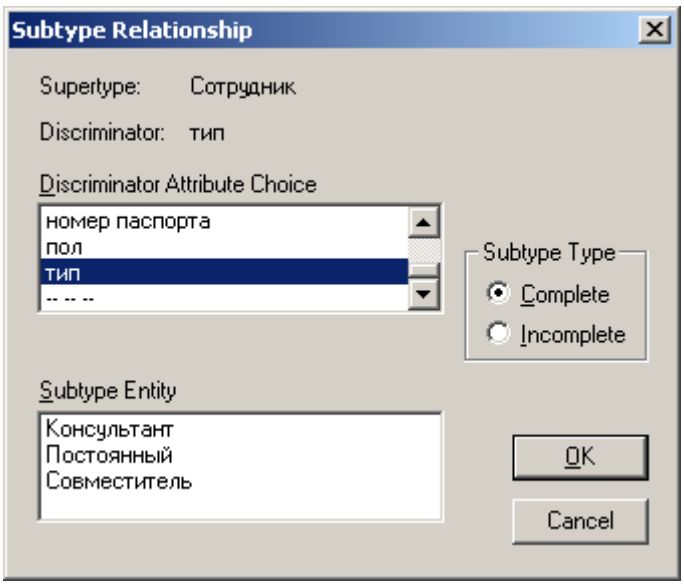


Рис. 31. Диалог спецификации свойств категориальной связи

При задании рекурсивной связи, определенной на сущности «Сотрудник», первичный ключ («табельный номер») должен мигрировать в качестве внешнего ключа в состав не ключевых атрибутов той же сущности. Поскольку один атрибут не может появиться дважды под одним и тем же именем, внешнему ключу должно быть присвоено имя роли.

Имя роли (функциональное имя) – это синоним атрибута внешнего ключа, который показывает, какую роль играет внешний ключ в моделировании связи.

Определение имени роли производится в диалоге **Relationships** спецификации свойств связи (рис. 32). Для спецификации свойств связи (в частности унарной рекурсивной связи «Руководит/Подчиняется») следует щелкнуть правой кнопкой мыши по ее шаблону и выбирать в контекстном меню пункт **Relationship Properties.** В диалоге **Relationships** на закладке **Rolename** в окно Rolename следует ввести имя роли, например, «начальник». По умолчанию в списке атрибутов показывается только имя роли. Для отображения полного имени атрибута следует в контекстном меню, которое появляется, если щелкнуть правой кнопкой мыши по любому месту диаграммы, не занятому объектами модели, выбрать пункт **Entitiy Display** и затем включить опцию **Rolename/Attribute**. Полное имя показывается как функциональное имя и базовое имя, разделенные точкой (рис. 30).

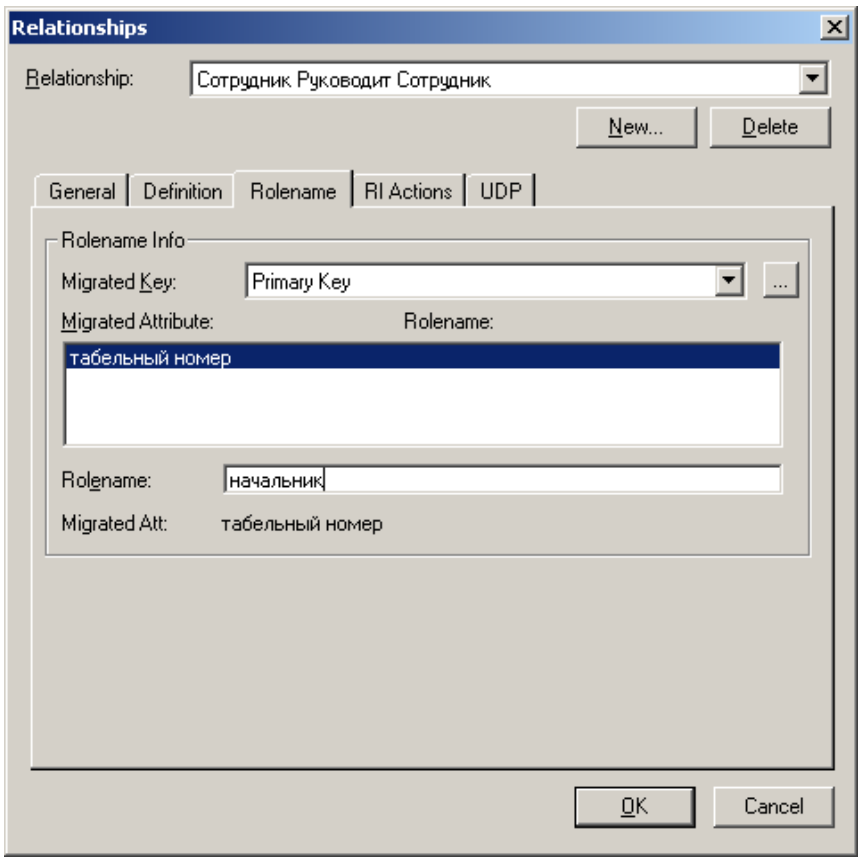


Рис. 32. Определение имени роли внешнего ключа

ЗАКЛЮЧЕНИЕ

Итак, мы построили полную атрибутивную модель учебной предметной области, познакомились с методологией моделирования сущностей и связей и технологией работы в среде ERwin. Таким образом, главная цель учебного пособия достигнута. Полученные результаты рекомендуется осмыслить и сохранить для продолжения работы. Построенная модель отображает наше представление о структуре данных, выраженное в терминах модели «сущность – связь» в нотации IDEF1X. Однако разработка информационной системы не заканчивается построением инфологической модели предметной области. Особенность методологии IDEF1X такова, что она предполагает отображение логической ER-модели на реляционную модель данных с последующей генерацией схемы базы данных. В CASE-среде ERwin работа по проектированию информационной системы может быть продолжена и доведена до логического конца. Но это уже другая проблема, которая требует более детального изучения.

БИБЛИОГРАФИЧЕСКИЙ СПИСОК

1. **Вендров, А.М.** CASE-технологии. Современные методы и средства проектирования информационных систем [Электронный ресурс] /А.М. Вендров. – Режим доступа: http://case-tech.h1.ru/library/vendrov/index.htm.

2. **Дейт, К.Дж.** Введение в системы баз данных [Текст] / К.Дж. Дейт ; пер. с англ. – 6-е изд. – Киев ; М. ; СПб. : Издательский дом «Вильямс», 1999. – 848 с.

3. **Хомоненко, А.Д.** Базы данных: учебник для высших учебных заведений [Текст] / А.Д. Хомоненко, В.М. Цыганов, М.Г. Мальцев ; под ред. проф. А.Д. Хомоненко. – изд. 2-е, доп. и перераб. – СПб. : КОРОНА принт, 2002. – 672 с.

4. **Коннолли, Т.** Базы данных: проектирование, реализация и сопровождение. Теория и практика [Текст] / Т. Коннолли, К. Бегг, А. Страчан ; пер. с англ. – 2-е изд. – М. : Издательский дом «Вильямс», 2000. – 1120 с.

5. **Чен, П.П-Ш.** Модель «сущность – связь» – шаг к единому представлению о данных [Электронный ресурс] / П.П-Ш. Чен ; пер. М.Р. Когаловской. – Режим доступа: http://citforum.ru/database/classics/chen/.

6. **Смит, Д.М.** Абстракции баз данных: агрегация и обобщение [Электронный ресурс] / Д.М. Смит, Д.К. Смит, пер. М.Р. Когаловской. – Режим доступа: http://citforum.ru/database/classics/abstraction_generalization/.

7. **Грошев, А.С.** Использование методологии IDEF1X для разработки концептуальной модели данных [Электронный ресурс] / А.С. Грошев. Основы работы с базами данных: курс лекций. – Режим доступа: http://www.intuit.ru/department/database/basedbw/3/.

8. **Верников, Г.** Основы методологии IDEF1X [Электронный ресурс]. – Режим доступа: http://www.interface.ru/fset.asp?Url=/ca/idef1x.htm.

9. **Маклаков, С.В.** Создание информационных систем с AllFusion Modeling Suite [Электронный ресурс]. – Режим доступа: http://biblioclub.ru/index.php?page=book&id=54771.

10. **Кара-Ушанов, В.Ю.** Разработка баз данных в CASE-среде Erwin: учебное пособие [Текст] / В.Ю. Кара-Ушанов. – Екатеринбург : Екатеринбургская академия современного искусства, 2011. – 134 с.

11. **Этлингер, Б.** Первые шаги с ERwin 4.0 (Серия статей) [Электронный ресурс]. – Режим доступа: http://www.interface.ru/fset.asp?Url=/ca/erwin.htm.

12. Руководство по программному пакету ERwin [Электронный ресурс]. – Режим доступа: http://www.xserver.ru/computer/database/erwin/2/index.shtml.

13. **Грекул, В.** Лекция 10. Моделирование информационного обеспечения [Электронный ресурс] / В. Грекул. Проектирование информационных систем. – Режим доступа: http://www.intuit.ru/studies/courses/2195/55/lecture/1636?page=1.

Учебное электронное текстовое издание

Кара-Ушанов Владимир Юрьевич

МОДЕЛЬ «СУЩНОСТЬ – СВЯЗЬ»

Учебное пособие

Редактор *А.В. Ерофеева*
Подготовка к публикации *А.В. Ерофеевой*

Разрешено к публикации 03.07.2017
Электронный формат – pdf
Объем 1,8 уч.-изд. л.

Уральский федеральный университет

620002, Екатеринбург, ул. Мира, 19

Информационный портал УрФУ
http://www.urfu.ru